双臂复合升降ROS2-foxy-moveit2配置教程

1. 准备工作：

- 操作系统：ubuntu22.04、ubuntu20.04
- ROS：humble、foxy
- URDF：双臂复合升降机器人的URDF

声明：

- 作者：Starry；有什么问题请随时联系我更正。
- 邮箱：2308607900@qq.com

好废话不多说，直接开始，这里解释一下为什么用到两个`ubuntu`系统和两个`ROS`版本，是因为`foxy`版本的二进制安装`moveit2`的时候是没有`moveit-setup-assistant`配置助手，所以我们使用`humble`配置好之后迁移到`foxy`版本，如果您觉得这种方式太繁琐，那么可以通过源码安装`moveit2`从而实现在`foxy`版本上直接配置。这里就不介绍怎么使用源码安装的方式了，本人有两个系统。

2. 新建机器人描述功能包

先进入ubuntu22.04中，新建工作空间，并且新建功能包

```
# 新建工作空间
mkdir -p urdf_moveit/src && cd urdf_moveit/src/
# 工作空间的src目录下新建功能包
ros2 pkg create dual_arm_description --build-type ament_cmake
# 到功能包目录下新建目录 urdf目录就存放urdf文件 launch目录存放launch文件 meshes目录存放网格描述文件
cd dual_arm_description/ && mkdir launch meshes urdf
```

最终的目录结构：

```
.
├── CMakeLists.txt
├── include
│   └── dual_arm_description
├── launch
│   └── display.launch.py
├── meshes
│   ├── base_link_agv.STL
│   ├── base_link_body.STL
│   ├── base_link_leftarm.STL
│   ├── base_link_platform.STL
│   ├── base_link_rightarm.STL
│   ├── leftarm_Link1.STL
│   ├── leftarm_Link2.STL
│   ├── leftarm_Link3.STL
│   ├── leftarm_Link4.STL
│   ├── leftarm_Link5.STL
│   ├── leftarm_Link6.STL
```

```
|   ├── leftarm_Link7.STL
|   ├── lefthand_Link.STL
|   ├── Link_camera.STL
|   ├── Link_head.STL
|   ├── Link_left_wheel.STL
|   ├── Link_radar.STL
|   ├── Link_right_wheel.STL
|   ├── rightarm_Link1.STL
|   ├── rightarm_Link2.STL
|   ├── rightarm_Link3.STL
|   ├── rightarm_Link4.STL
|   ├── rightarm_Link5.STL
|   ├── rightarm_Link6.STL
|   ├── rightarm_Link7.STL
|   └── righthand_Link.STL
├── package.xml
├── src
└── urdf
    ├── dual_arm_75.urdf.xacro
    ├── dual_arm_description.csv
    └── dual_arm_description.urdf
```

`display.launch.py`文件内容如下：

```python
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, ExecuteProcess,
IncludeLaunchDescription
from launch.conditions import IfCondition
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration, PythonExpression
from launch_ros.actions import Node

def generate_launch_description():
    # Get the launch directory
    bringup_dir = get_package_share_directory('dual_arm_description')
    launch_dir = os.path.join(bringup_dir, 'launch')

    # Launch configuration variables specific to simulation
    rviz_config_file = LaunchConfiguration('rviz_config_file')
    use_robot_state_pub = LaunchConfiguration('use_robot_state_pub')
    use_joint_state_pub = LaunchConfiguration('use_joint_state_pub')
    use_rviz = LaunchConfiguration('use_rviz')
    urdf_file= LaunchConfiguration('urdf_file')

    declare_rviz_config_file_cmd = DeclareLaunchArgument(
        'rviz_config_file',
        default_value=os.path.join(bringup_dir, 'rviz', 'view.rviz'),
        description='Full path to the RVIZ config file to use')
    declare_use_robot_state_pub_cmd = DeclareLaunchArgument(
        'use_robot_state_pub',
        default_value='True',
        description='Whether to start the robot state publisher')
    declare_use_joint_state_pub_cmd = DeclareLaunchArgument(
```

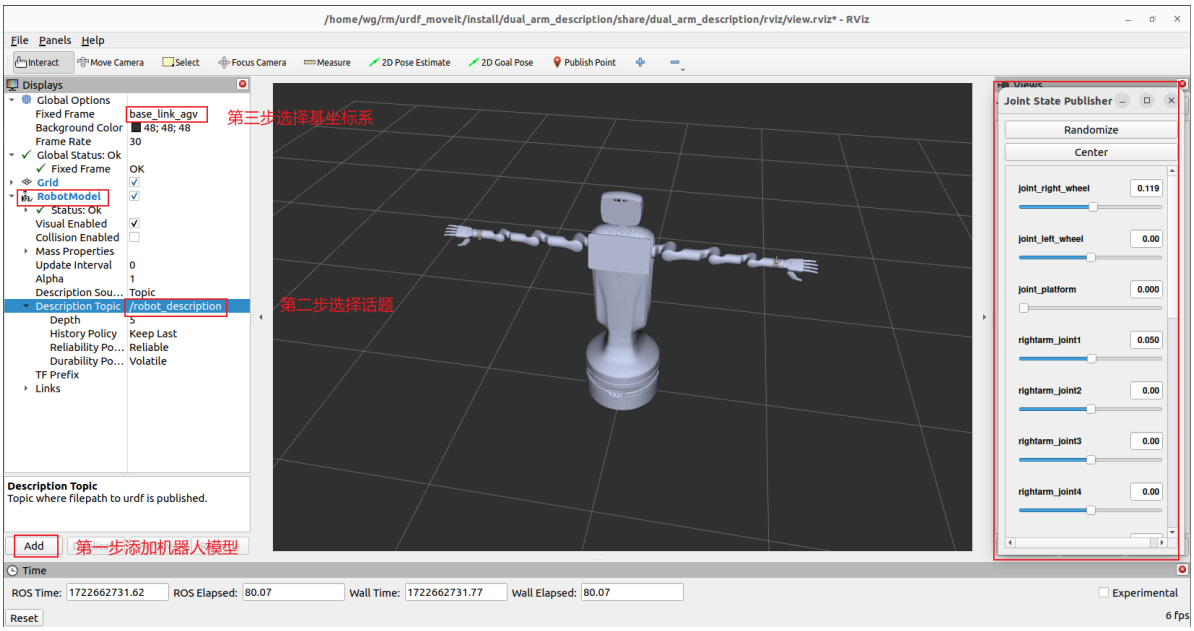
```
        'use_joint_state_pub',
        default_value='True',
        description='Whether to start the joint state publisher')
    declare_use_rviz_cmd = DeclareLaunchArgument(
        'use_rviz',
        default_value='True',
        description='Whether to start RVIZ')

    declare_urdf_cmd = DeclareLaunchArgument(
        'urdf_file',
        default_value=os.path.join(bringup_dir, 'urdf',
'dual_arm_description.urdf'),
        description='Whether to start RVIZ')

    start_robot_state_publisher_cmd = Node(
        condition=IfCondition(use_robot_state_pub),
        package='robot_state_publisher',
        executable='robot_state_publisher',
        name='robot_state_publisher',
        output='screen',
        #parameters=[{'use_sim_time': use_sim_time}],
        arguments=[urdf_file])

    start_joint_state_publisher_cmd = Node(
        condition=IfCondition(use_joint_state_pub),
        package='joint_state_publisher_gui',
        executable='joint_state_publisher_gui',
        name='joint_state_publisher_gui',
        output='screen',
        arguments=[urdf_file])

    rviz_cmd = Node(
        condition=IfCondition(use_rviz),
        package='rviz2',
        executable='rviz2',
        name='rviz2',
        arguments=['-d', rviz_config_file],
        output='screen')

    # Create the launch description and populate
    ld = LaunchDescription()

    # Declare the launch options
    ld.add_action(declare_rviz_config_file_cmd)
    ld.add_action(declare_urdf_cmd)
    ld.add_action(declare_use_robot_state_pub_cmd)
    ld.add_action(declare_use_joint_state_pub_cmd)
    ld.add_action(declare_use_rviz_cmd)

    # Add any conditioned actions
    ld.add_action(start_joint_state_publisher_cmd)
    ld.add_action(start_robot_state_publisher_cmd)
    ld.add_action(rviz_cmd)

    return ld
```

修改`CMakeLists.txt`

```
install(DIRECTORY launch urdf meshes  DESTINATION share/${PROJECT_NAME})
```

在工作空间目录下编译功能包并测试：

```
# 编译
colcon build --packages-select dual_arm_description
# 在rviz2中显示机器人模型
source install/setup.bash
ros2 launch dual_arm_description display.launch.py
```



右边的滑动小窗口可以拖动滑条查看各个关节是否符合预期。

3. moveit-setup-assistant配置

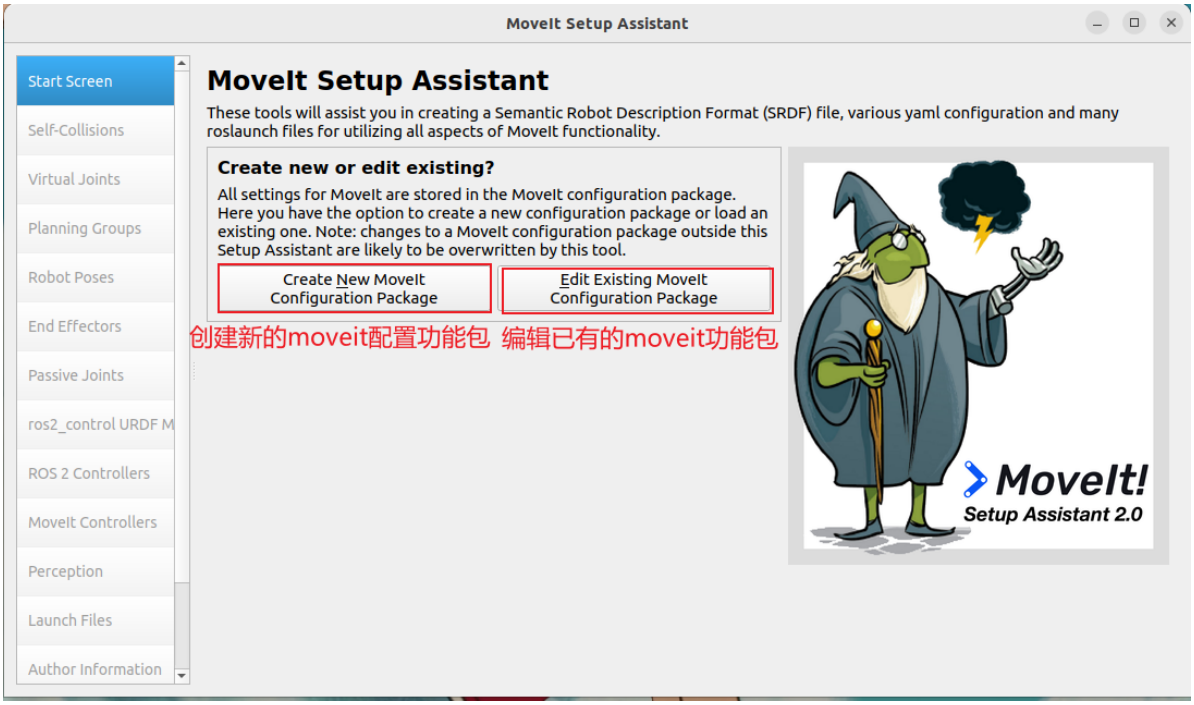
3.1 安装moveit2和配置助手

```
sudo apt install  ros-humble-moveit ros-humble-moveit-setup-assistant -y
```

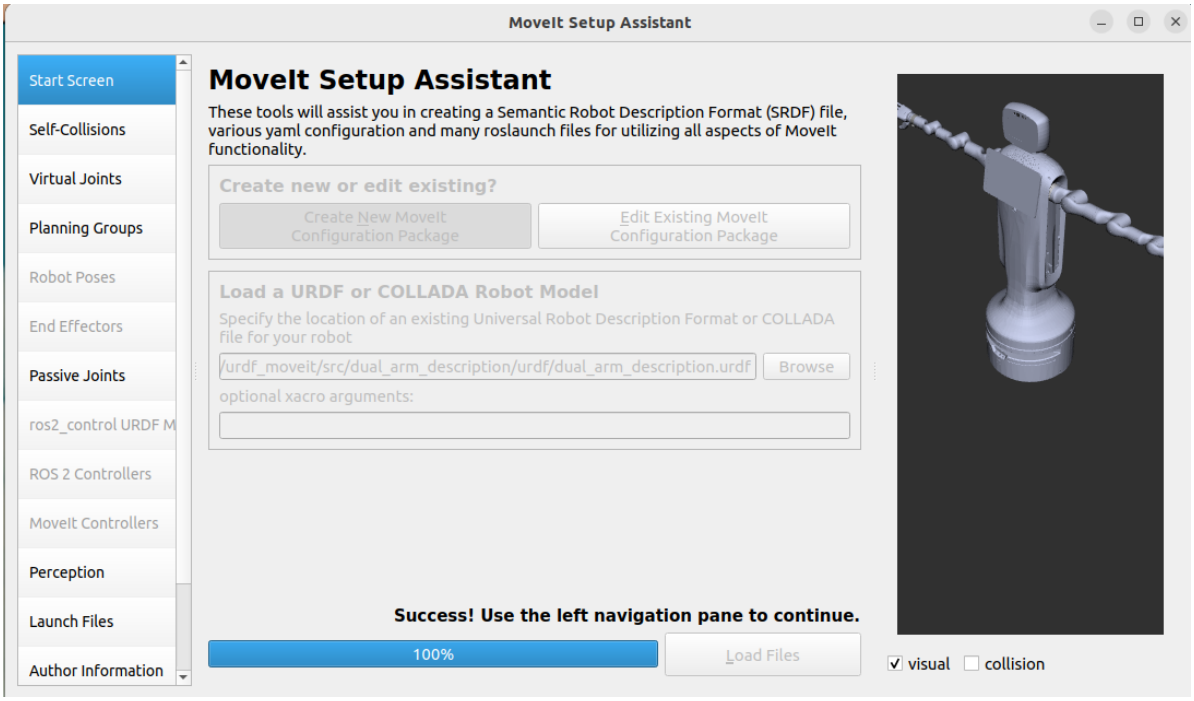
3.2 启动配置助手

到工作空间目录下

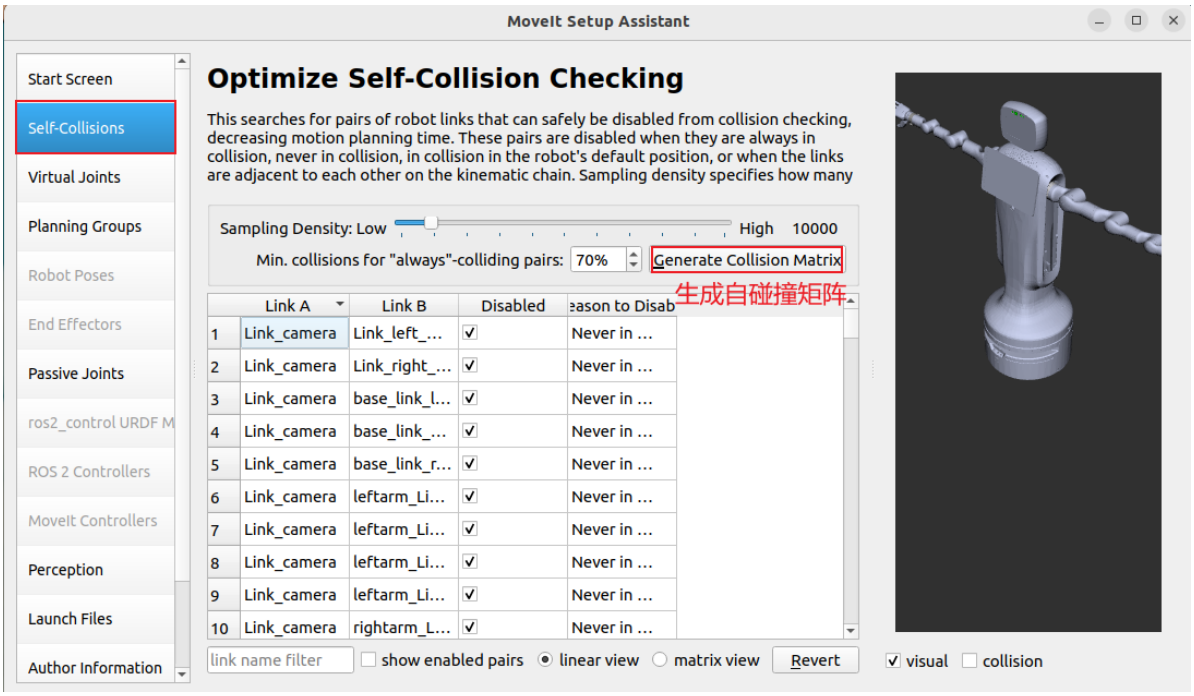
```
# 这里为什么需添加一下工作空间的环境变量呢，是防止避免新开终端的时候启动
moveit_setup_assistant配置助手找不到meshes文件
source install/setup.bash
ros2 run moveit_setup_assistant moveit_setup_assistant
```



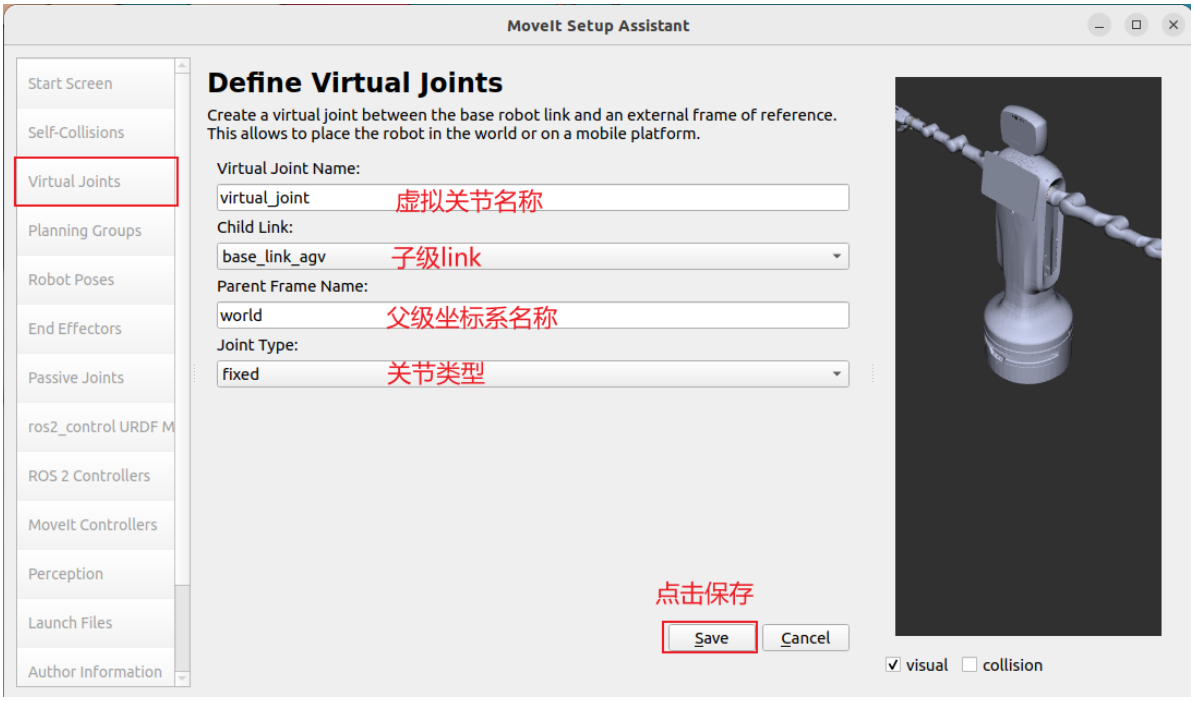
3.3 选择创建新的moveit配置功能包



3.4 生成自碰撞矩阵

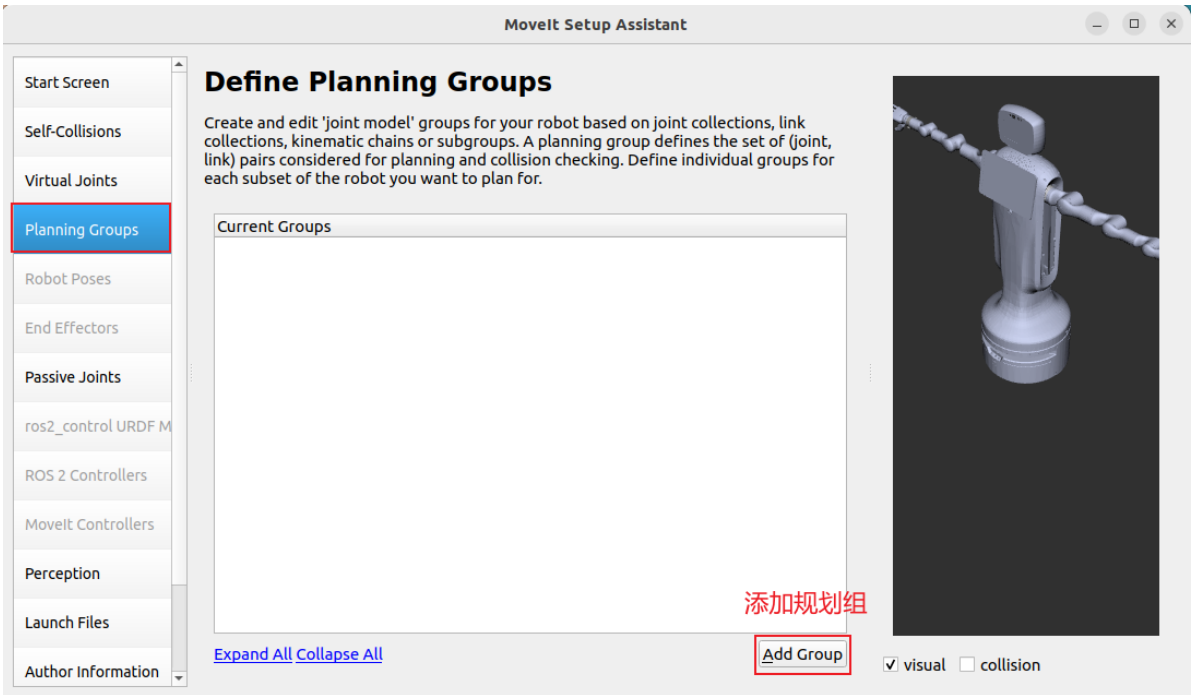


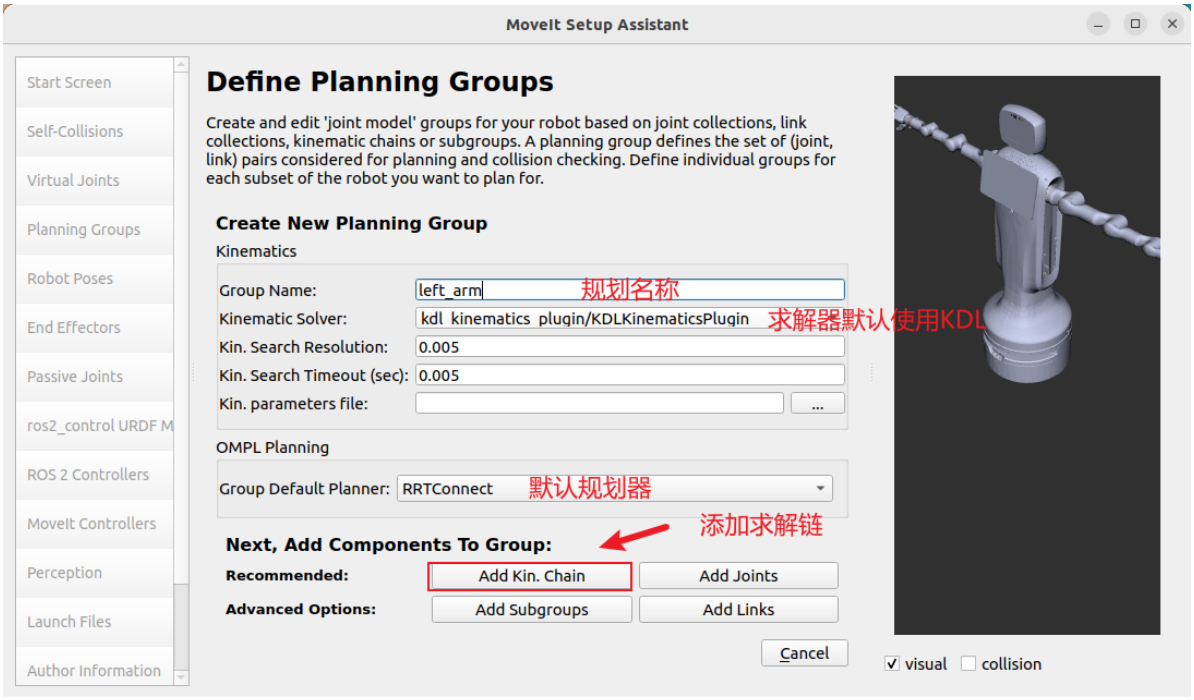
3.5 添加虚拟关节



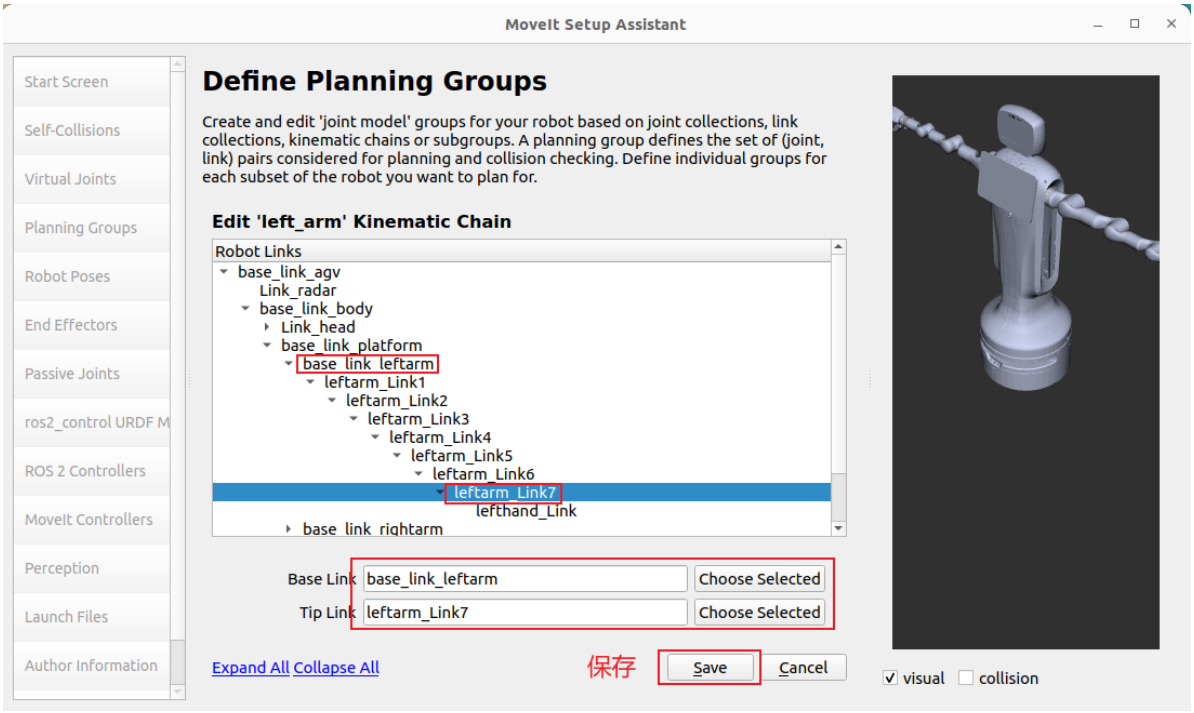
3.6 添加规划组

3.6.1 添加左臂规划组

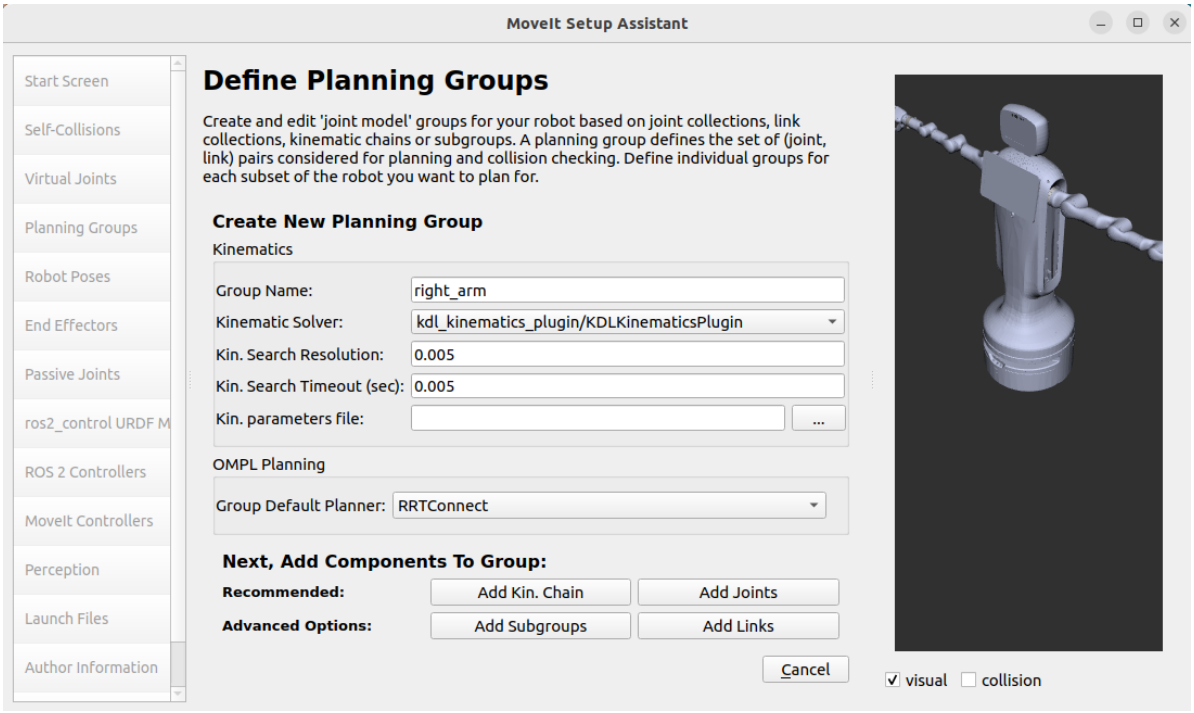




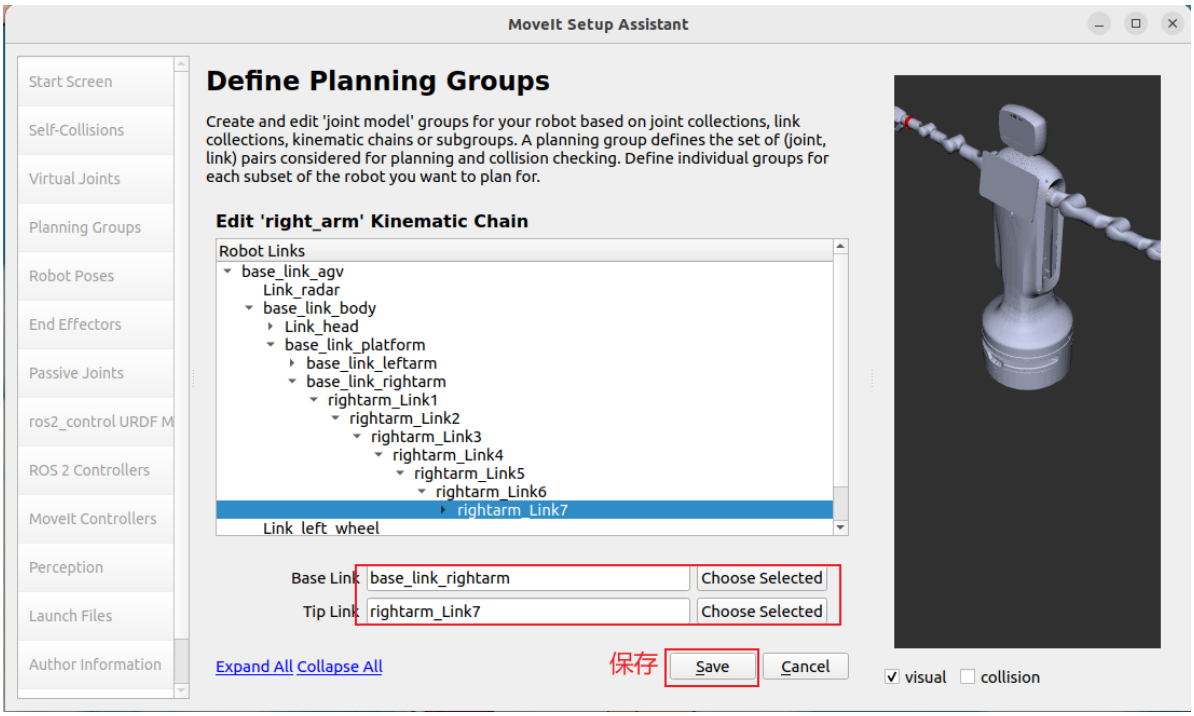
3.6.2 添加左臂的求解链



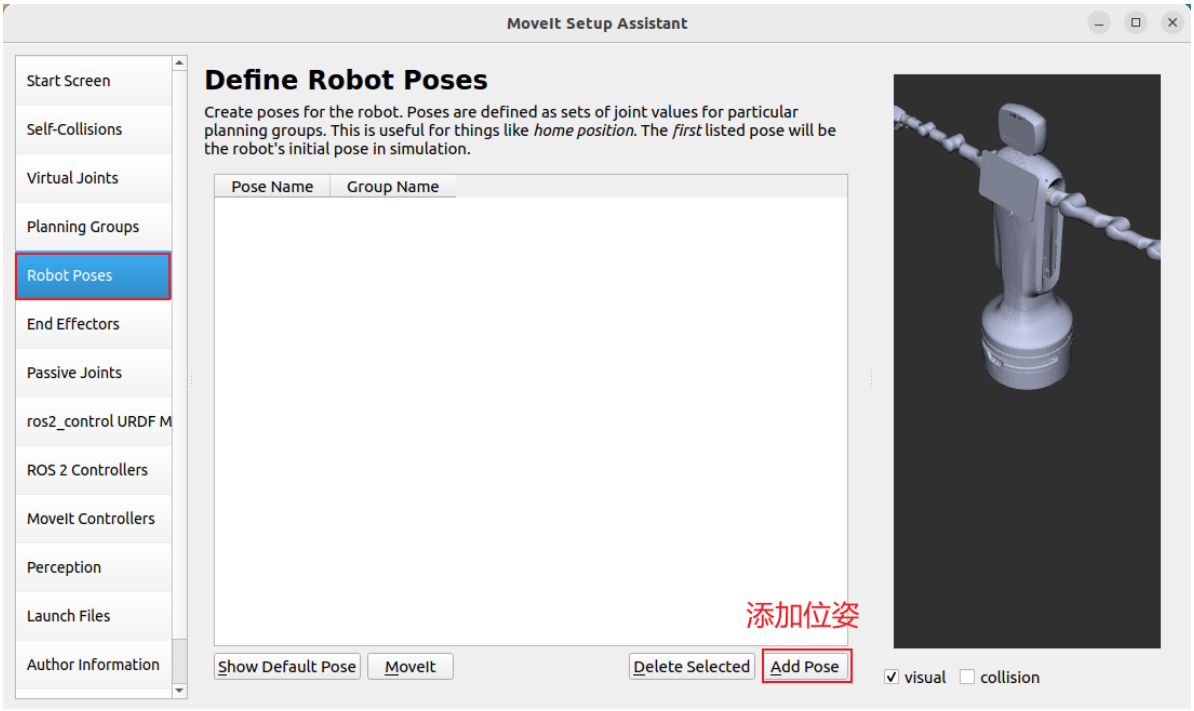
3.6.3 添加右臂规划组

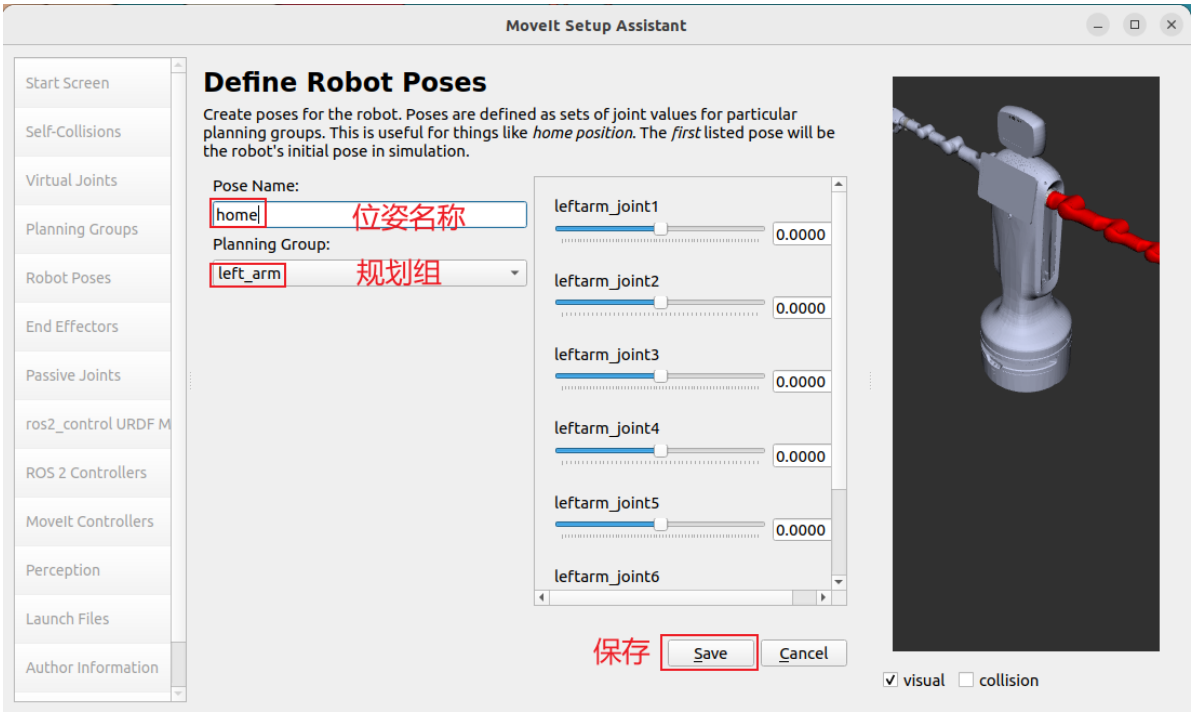


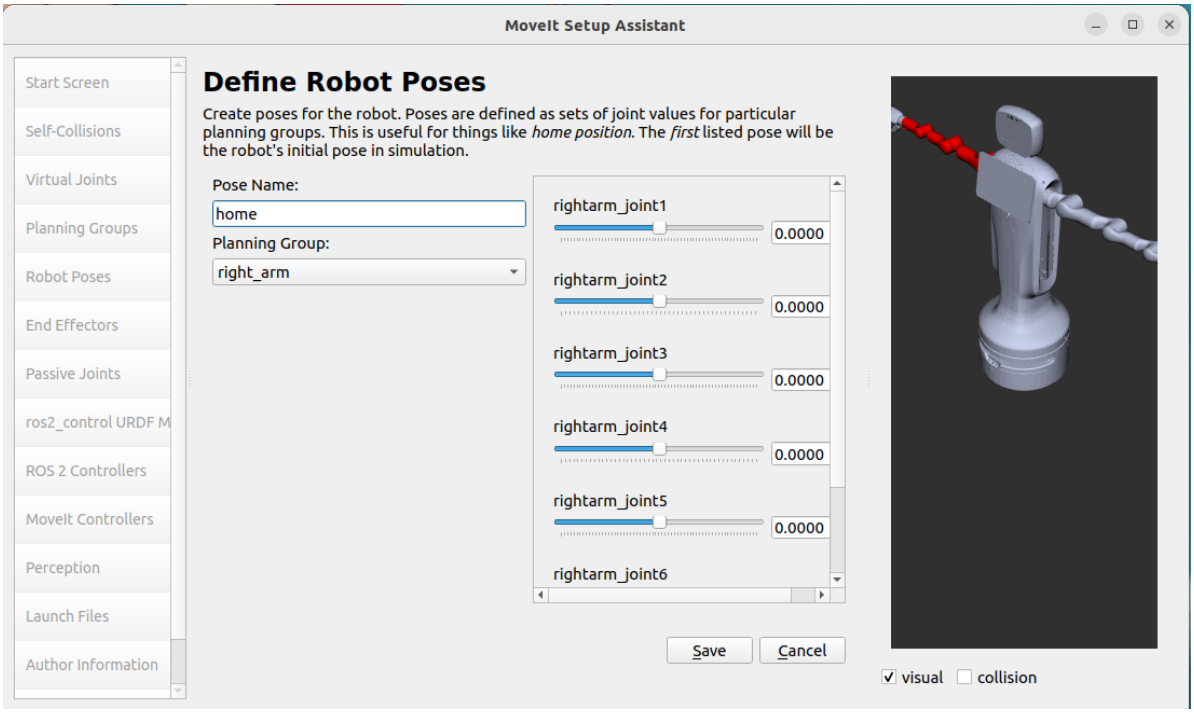
3.6.4 添加右臂的求解链

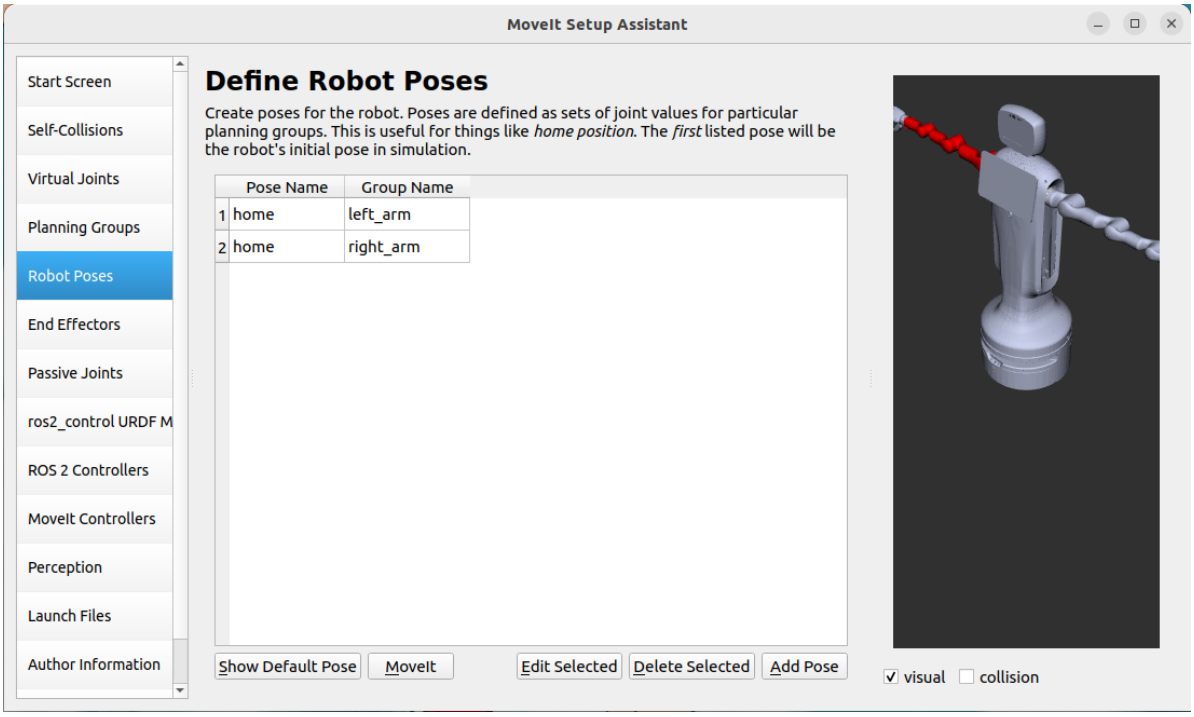


3.7 添加机器人位姿

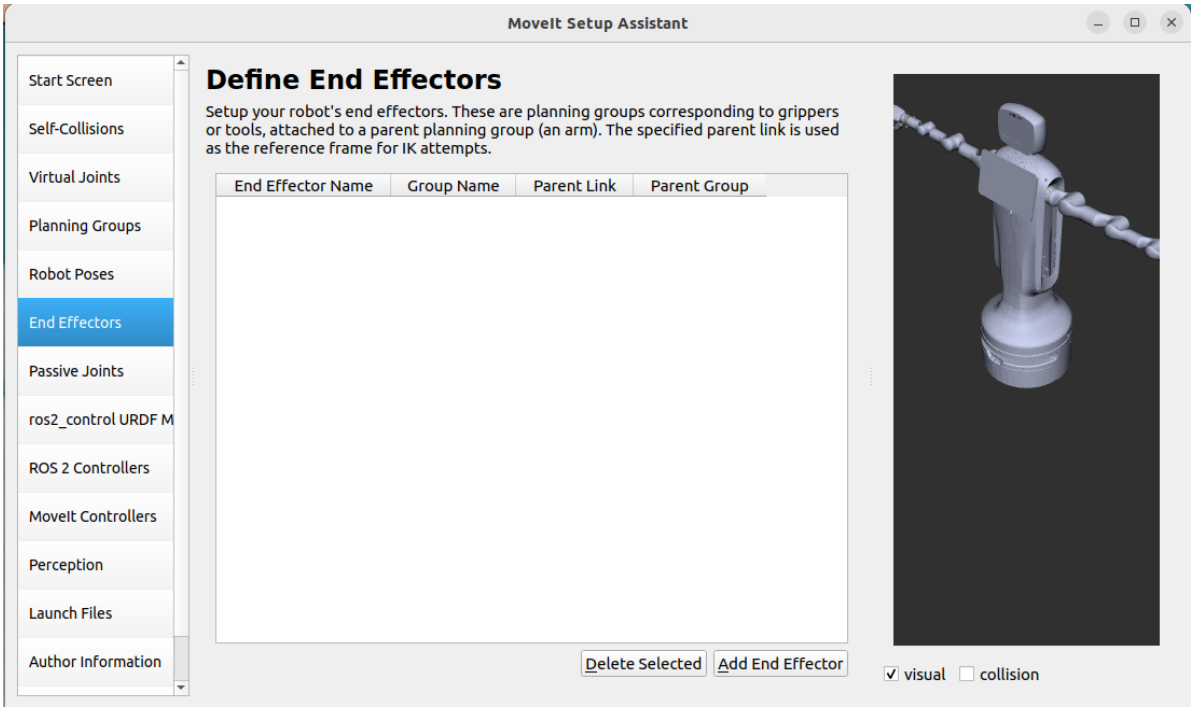

MoveIt Setup Assistant
Start Screen
Self-Collisions
Virtual Joints
Planning Groups
Robot Poses
End Effectors
Passive Joints
ros2_control URDF M
ROS 2 Controllers
MoveIt Controllers
Perception
Launch Files
Author Information
Define Robot Poses
Create poses for the robot. Poses are defined as sets of joint values for particular planning groups. This is useful for things like home position. The first listed pose will be the robot's initial pose in simulation.
Pose Name
Group Name
添加位姿
Show Default Pose
MoveIt
Delete Selected
Add Pose
visual
collision


MoveIt Setup Assistant
Start Screen
Self-Collisions
Virtual Joints
Planning Groups
Robot Poses
End Effectors
Passive Joints
ros2_control URDF M
ROS 2 Controllers
MoveIt Controllers
Perception
Launch Files
Author Information
Define Robot Poses
Create poses for the robot. Poses are defined as sets of joint values for particular planning groups. This is useful for things like home position. The first listed pose will be the robot's initial pose in simulation.
Pose Name:
home
位姿名称
Planning Group:
left_arm
规划组
leftarm_joint1
leftarm_joint2
leftarm_joint3
leftarm_joint4
leftarm_joint5
leftarm_joint6
0.0000
保存
Save
Cancel
visual
collision


MoveIt Setup Assistant
Start Screen
Self-Collisions
Virtual Joints
Planning Groups
Robot Poses
End Effectors
Passive Joints
ros2_control URDF M
ROS 2 Controllers
MoveIt Controllers
Perception
Launch Files
Author Information
Define Robot Poses
Create poses for the robot. Poses are defined as sets of joint values for particular planning groups. This is useful for things like home position. The first listed pose will be the robot's initial pose in simulation.
Pose Name:
home
Planning Group:
right_arm
rightarm_joint1
0.0000
rightarm_joint2
0.0000
rightarm_joint3
0.0000
rightarm_joint4
0.0000
rightarm_joint5
0.0000
rightarm_joint6
Save
Cancel
visual
collision

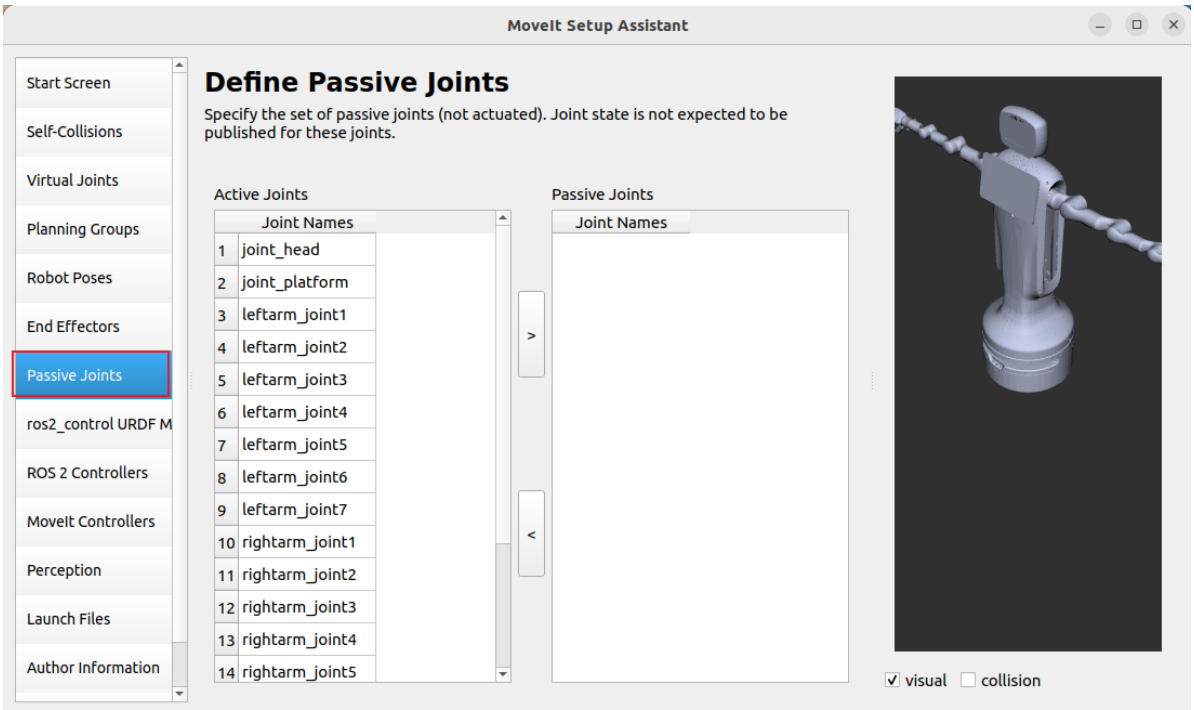

MoveIt Setup Assistant
Start Screen
Self-Collisions
Virtual Joints
Planning Groups
Robot Poses
End Effectors
Passive Joints
ros2_control URDF M
ROS 2 Controllers
MoveIt Controllers
Perception
Launch Files
Author Information
Define Robot Poses
Create poses for the robot. Poses are defined as sets of joint values for particular planning groups. This is useful for things like home position. The first listed pose will be the robot's initial pose in simulation.
Pose Name
Group Name
1 home left_arm
2 home right_arm
Show Default Pose
MoveIt
Edit Selected
Delete Selected
Add Pose
visual
collision

3.8 添加末端执行器



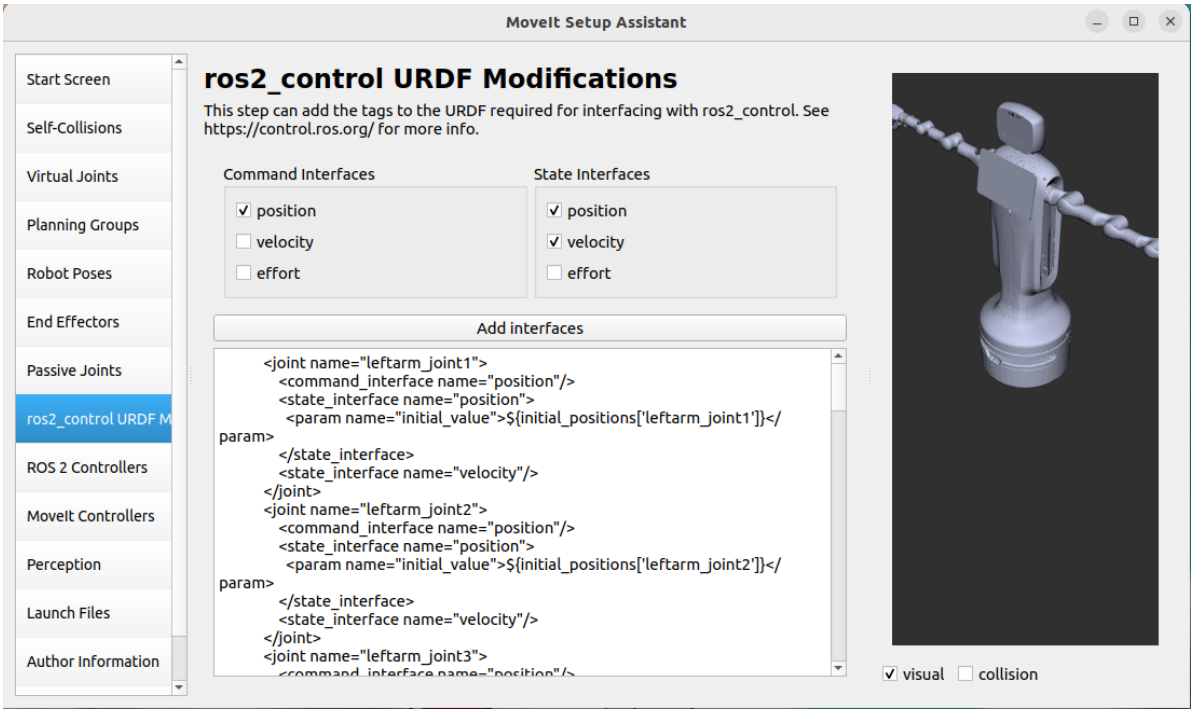
我这里没有就不用添加了，根据需求自行添加

3.9 添加固定关节

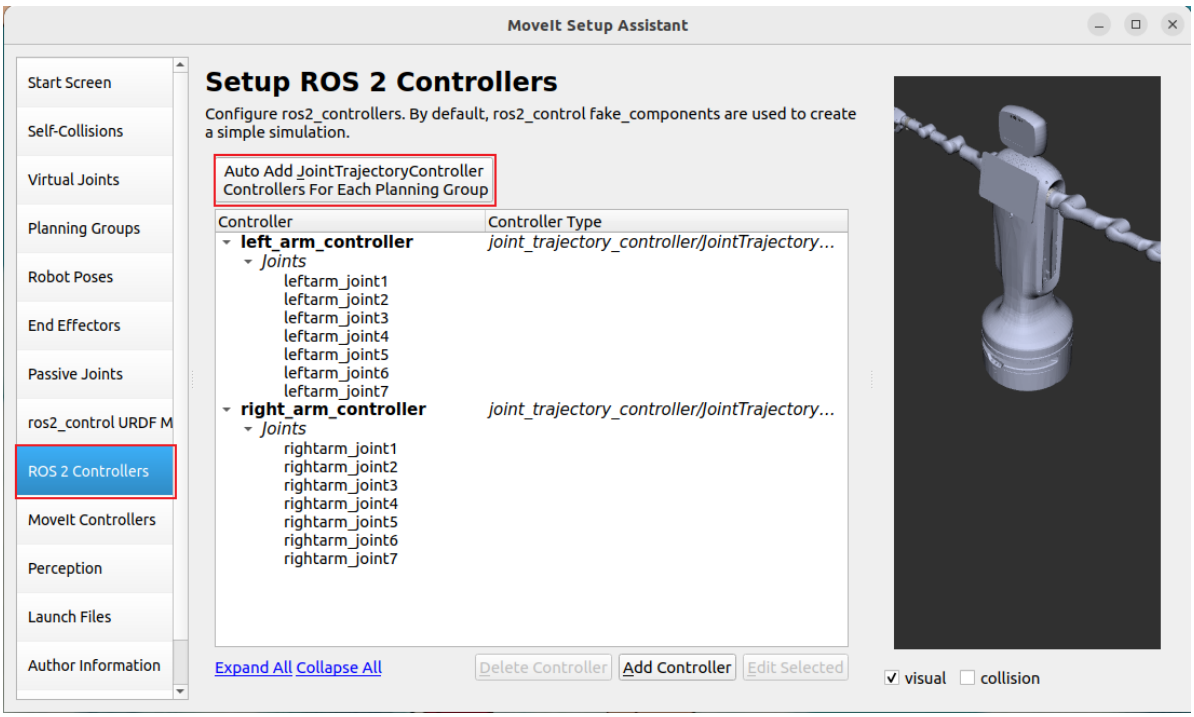


我这里没有就不添加了，同样根据需求添加

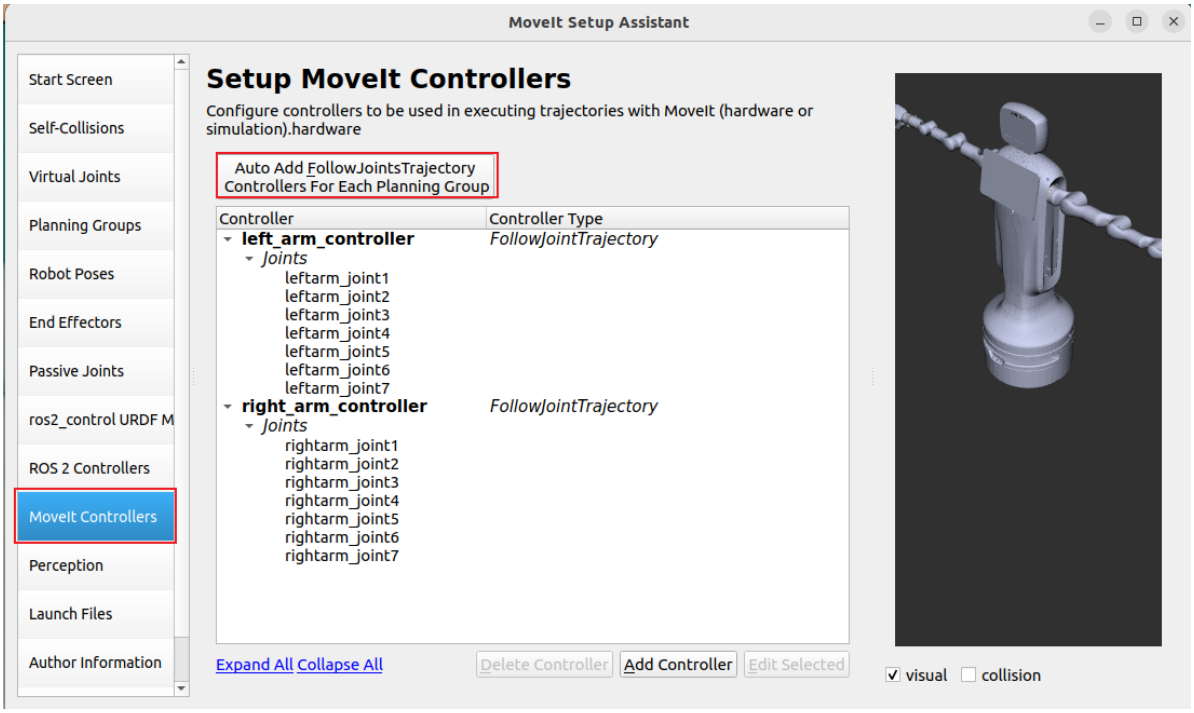
3.10 添加ros2_control接口



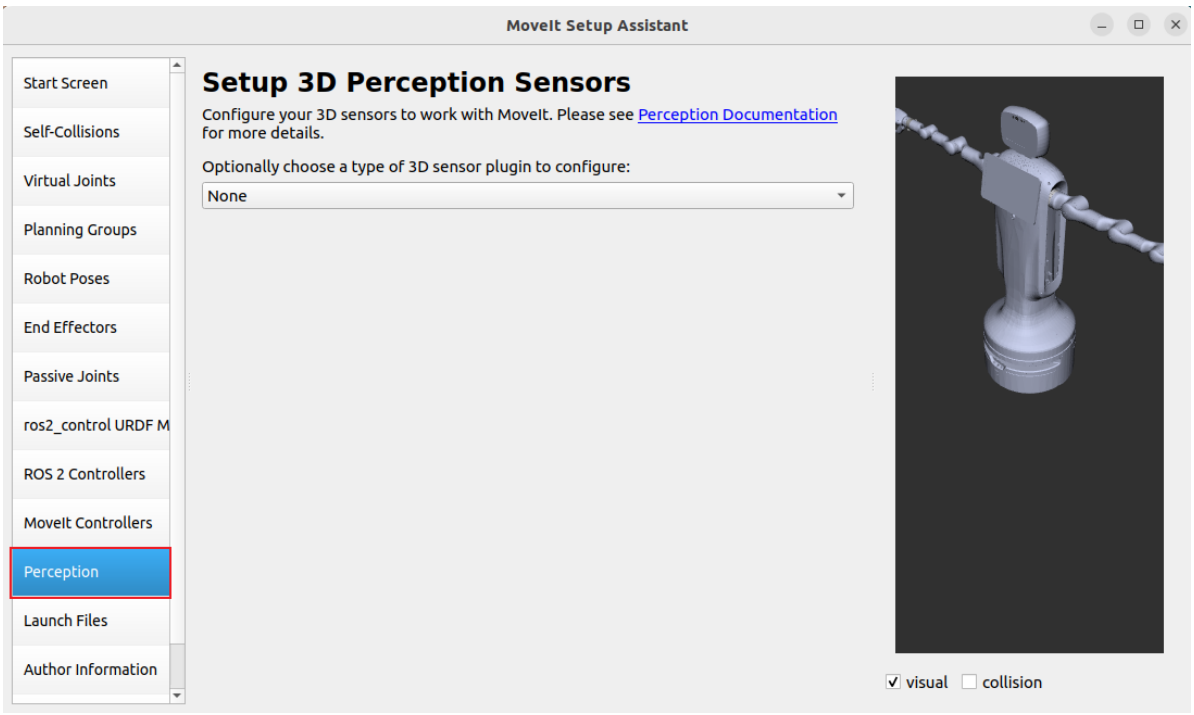
3.11 添加ros2 controllers



3.12 添加moveit controllers

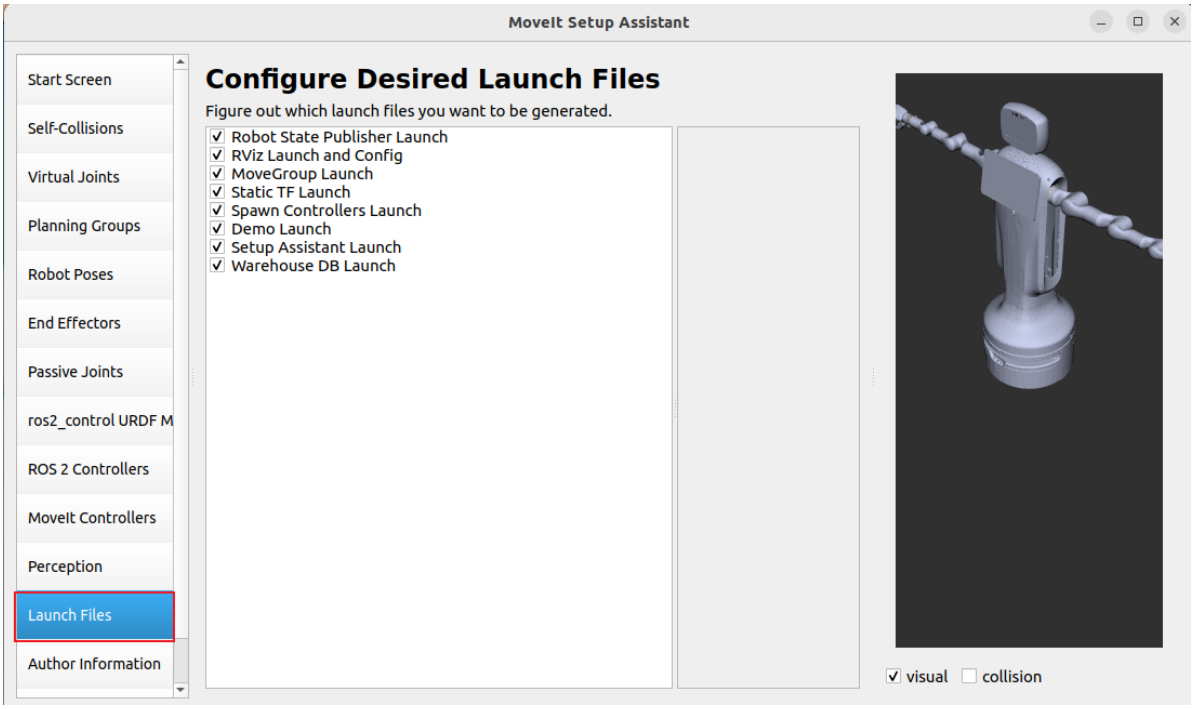


3.13 添加传感器

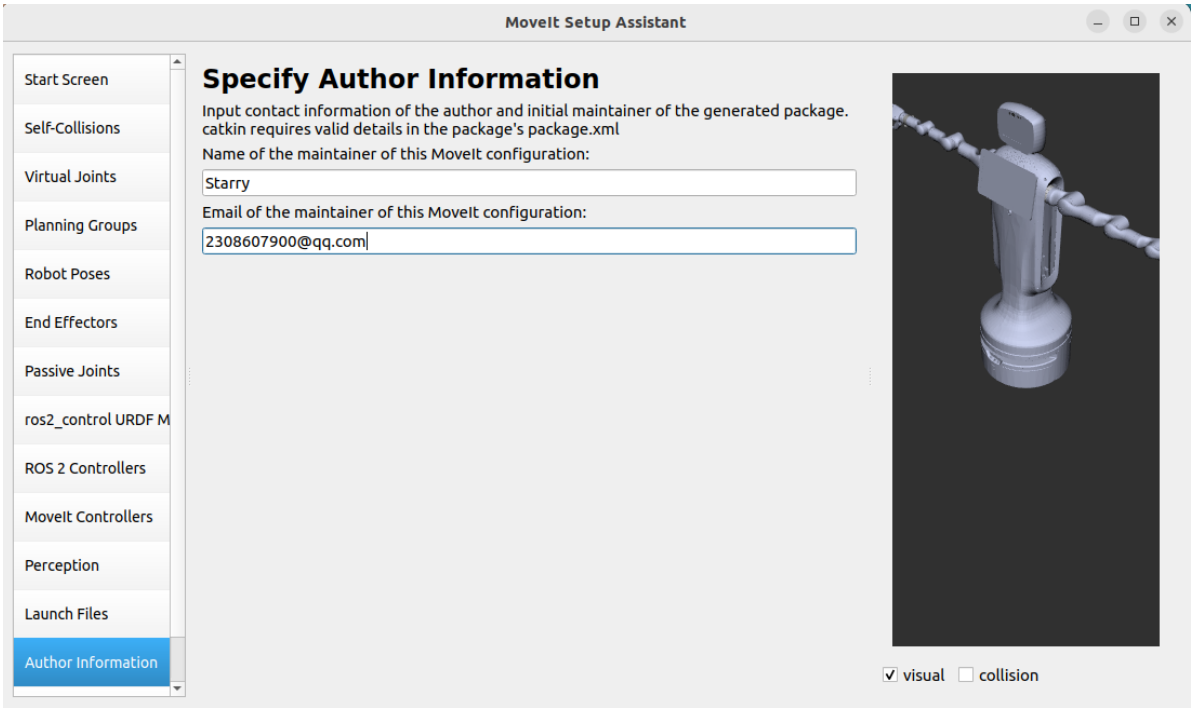


这里没有就不添加，有需要自行添加

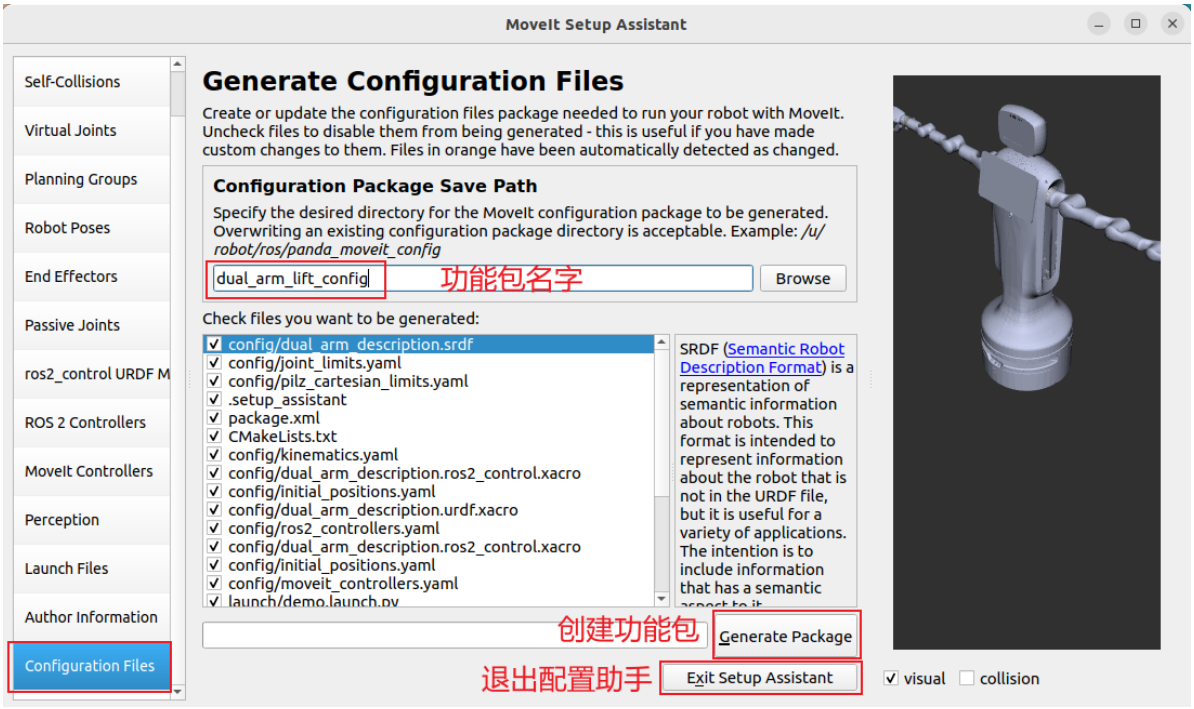
3.14 选择需要生成的launch文件



3.15 作者信息



3.16 创建功能包



创建完成功能包之后就可以退出配置助手了。这里注意下，生成功能包的目录默认是启动配置助手的目录。

ok，现在功能包已经生成，需要拷贝到ubuntu20.04中，准备u盘或者拷贝到Windows中，我这里使用的是虚拟机就拷贝到Windows中然后再拷贝到ubuntu20.04中。

4. 降低cmake版本

咱们得功能包是在Ubuntu22.04中创建生成的放到Ubuntu20.04中直接编译会报错

查看Ubuntu20.04的cmake版本

```
cmake --version
```

```
linx@linx-vm:~$ cmake --version
cmake version 3.16.3
```

打开(O)

```
1 cmake_minimum_required(VERSION 3.16)
2 project(dual_arm_lift_config)
3
4 find_package(ament_cmake REQUIRED)
5
6 ament_package()
7
8 install(DIRECTORY launch DESTINATION share/${PROJECT_NAME}
9   PATTERN "setup_assistant.launch" EXCLUDE)
10 install(DIRECTORY config DESTINATION share/${PROJECT_NAME})
11 install(FILES .setup_assistant DESTINATION share/${PROJECT_NAME})
```

```
1 cmake_minimum_required(VERSION 3.16)
2 project(dual_arm_description)
3
```

最后别忘记保存

到工作空间下编译

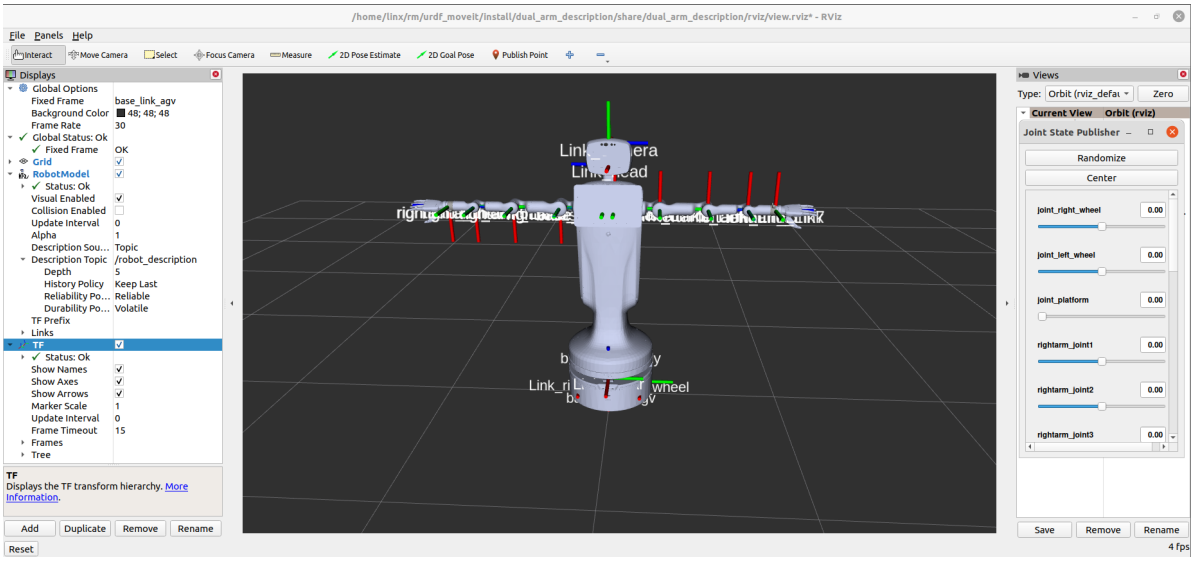
```
colcon build
```

```
linx@linx-vm:~/rm/urdf_moveit$ colcon build
Starting >>> dual_arm_description
Finished <<< dual_arm_description [5.21s]
Starting >>> dual_arm_lift_config
Finished <<< dual_arm_lift_config [1.06s]

Summary: 2 packages finished [7.85s]
linx@linx-vm:~/rm/urdf_moveit$
```

5. 显示机器人模型

先测试一下模型能不能正常显示：

```
source install/setup.bash
ros2 launch dual_arm_description display.launch.py
```



OK，可以看到模型成功显示。

6. 修改moveit配置功能包

6.1 修改`demo.launch.py`

这里解释一下为什么整个demo.launch.py都需要改，是因为配置直接使用moveit配置助手配置出来的不可用，这里我们就分模块启动各个组件。

```
import os
import yaml
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, TimerAction
from launch.substitutions import LaunchConfiguration
from launch.conditions import IfCondition, UnlessCondition
from launch_ros.actions import Node
from launch.actions import ExecuteProcess
from ament_index_python.packages import get_package_share_directory
```

```
import xacro


def load_file(package_name, file_path):
    package_path = get_package_share_directory(package_name)
    absolute_file_path = os.path.join(package_path, file_path)

    try:
        with open(absolute_file_path, "r") as file:
            return file.read()
    except EnvironmentError:  # parent of IOError, OSError *and* WindowsError
where available
        return None


def load_yaml(package_name, file_path):
    package_path = get_package_share_directory(package_name)
    absolute_file_path = os.path.join(package_path, file_path)

    try:
        with open(absolute_file_path, "r") as file:
            return yaml.safe_load(file)
    except EnvironmentError:  # parent of IOError, OSError *and* WindowsError
where available
        return None


def generate_launch_description():

    # Command-line arguments
    tutorial_arg = DeclareLaunchArgument(
        "rviz_tutorial", default_value="False", description="Tutorial flag"
    )

    # planning_context
    robot_description_config = xacro.process_file(
        os.path.join(
            get_package_share_directory("dual_arm_lift_config"),
            "config",
            "dual_arm_description.urdf.xacro",
        )
    )

    robot_description = {"robot_description": robot_description_config.toxml()}

    robot_description_semantic_config = load_file(
        "dual_arm_lift_config", "config/dual_arm_description.srdf"
    )

    robot_description_semantic = {
        "robot_description_semantic": robot_description_semantic_config
    }

    kinematics_yaml = load_yaml(
        "dual_arm_lift_config", "config/kinematics.yaml"
```

```
    )
    robot_description_kinematics = {"robot_description_kinematics":
kinematics_yaml}

    joint_limits_yaml = load_yaml("dual_arm_lift_config",
"config/joint_limits.yaml")
    joint_limits = {'robot_description_planning': joint_limits_yaml}


    # Planning Functionality
    ompl_planning_pipeline_config = {
        "move_group": {
            "planning_plugin": "ompl_interface/OMPLPlanner",
            "request_adapters":
"""default_planner_request_adapters/AddTimeOptimalParameterization
default_planner_request_adapters/FixWorkspaceBounds
default_planner_request_adapters/FixStartStateBounds
default_planner_request_adapters/FixStartStateCollision
default_planner_request_adapters/FixStartStatePathConstraints""",
            "start_state_max_bounds_error": 0.1,
            "planning_time":10.0,
            "trajectory_execution/allowed_execution_duration_scaling":10
        }
    }
    ompl_planning_yaml = load_yaml(
        "dual_arm_lift_config", "config/ompl_planning.yaml"
    )
    ompl_planning_pipeline_config["move_group"].update(ompl_planning_yaml)

    # Trajectory Execution Functionality
    moveit_simple_controllers_yaml = load_yaml(
        "dual_arm_lift_config", "config/moveit_controllers.yaml"
    )

    moveit_controllers = {
        "moveit_simple_controller_manager": moveit_simple_controllers_yaml,
        "moveit_controller_manager":
"moveit_simple_controller_manager/MoveItSimpleControllerManager",
    }

    trajectory_execution = {
        "moveit_manage_controllers": True,
        "trajectory_execution.allowed_execution_duration_scaling": 1.2,
        "trajectory_execution.allowed_goal_duration_margin": 0.5,
        "trajectory_execution.allowed_start_tolerance": 0.01,
    }

    planning_scene_monitor_parameters = {
        "publish_planning_scene": True,
        "publish_geometry_updates": True,
        "publish_state_updates": True,
        "publish_transforms_updates": True,
    }

    # Start the actual move_group node/action server
```

```
    run_move_group_node = Node(
        package="moveit_ros_move_group",
        executable="move_group",
        output="screen",
        parameters=[
            robot_description,
            robot_description_semantic,
            kinematics_yaml,
            ompl_planning_pipeline_config,
            trajectory_execution,
            joint_limits,
            moveit_controllers,
            planning_scene_monitor_parameters,
        ],
    )

    # RViz
    rviz_base =
os.path.join(get_package_share_directory("dual_arm_lift_config"), "config")
    rviz_full_config = os.path.join(rviz_base, "moveit.rviz")


    rviz_node = Node(
        package="rviz2",
        executable="rviz2",
        name="rviz2",
        output="log",
        arguments=["-d", rviz_full_config],
        parameters=[
            robot_description,
            robot_description_semantic,
            ompl_planning_pipeline_config,
            kinematics_yaml,
        ],
    )

    delayed_rviz_node = TimerAction(
        period=40.0,
        actions=[rviz_node]
    )

    # Static TF
    static_tf = Node(
        package="tf2_ros",
        executable="static_transform_publisher",
        name="static_transform_publisher",
        output="log",
        arguments=["0.0", "0.0", "0.0", "0.0", "0.0", "0.0", "world",
"base_link"],
    )

    # Publish TF
    robot_state_publisher = Node(
        package="robot_state_publisher",
        executable="robot_state_publisher",
```

```
        name="robot_state_publisher",
        output="both",
        parameters=[robot_description],
    )

    # ros2_control using FakeSystem as hardware
    ros2_controllers_path = os.path.join(
        get_package_share_directory("dual_arm_lift_config"),
        "config",
        "ros2_controllers.yaml",
    )

    ros2_control_node = Node(
        package="controller_manager",
        executable="ros2_control_node",
        parameters=[robot_description, ros2_controllers_path],
        output={
            "stdout": "screen",
            "stderr": "screen",
        },
    )

    # Load controllers

    load_controllers = []
    for controller in [
        "left_arm_controller","right_arm_controller","joint_state_broadcaster"]:
        load_controllers += [
            ExecuteProcess(
                cmd=["ros2 run controller_manager spawner.py
{}".format(controller)],
                shell=True,
                output="screen",
            )
        ]

    # Warehouse mongodb server
    mongodb_server_node = Node(
       package="warehouse_ros_mongo",
       executable="mongo_wrapper_ros.py",
       parameters=[
           {"warehouse_port": 33829},
           {"warehouse_host": "localhost"},
           {"warehouse_plugin": "warehouse_ros_mongo::MongoDatabaseConnection"},
       ],
       output="screen",
    )

    return LaunchDescription(
        [
            tutorial_arg,
            delayed_rviz_node,
            static_tf,
            robot_state_publisher,
            run_move_group_node,
```

```
            ros2_control_node,
        #    mongodb_server_node,
        ]
        + load_controllers
    )
```

6.2 修改moveit_controllers.yaml

```yaml
controller_names:
  - left_arm_controller
  - right_arm_controller

left_arm_controller:
  type: FollowJointTrajectory
  action_ns: follow_joint_trajectory
  default: true
  joints:
    - leftarm_joint1
    - leftarm_joint2
    - leftarm_joint3
    - leftarm_joint4
    - leftarm_joint5
    - leftarm_joint6
    - leftarm_joint7
right_arm_controller:
  type: FollowJointTrajectory
  action_ns: follow_joint_trajectory
  default: true
  joints:
    - rightarm_joint1
    - rightarm_joint2
    - rightarm_joint3
    - rightarm_joint4
    - rightarm_joint5
    - rightarm_joint6
    - rightarm_joint7
```

6.3 修改`ros2_controllers.yaml`

```yaml
controller_manager:
  ros__parameters:
    update_rate: 100  # Hz

    left_arm_controller:
      type: joint_trajectory_controller/JointTrajectoryController
      action_ns: follow_joint_trajectory  # 添加这一行

    right_arm_controller:
      type: joint_trajectory_controller/JointTrajectoryController
      action_ns: follow_joint_trajectory  # 添加这一行
```

```
    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

left_arm_controller:
  ros__parameters:
    joints:
      - leftarm_joint1
      - leftarm_joint2
      - leftarm_joint3
      - leftarm_joint4
      - leftarm_joint5
      - leftarm_joint6
      - leftarm_joint7
    command_interfaces:
      - position
    state_interfaces:
      - position
      - velocity
right_arm_controller:
  ros__parameters:
    joints:
      - rightarm_joint1
      - rightarm_joint2
      - rightarm_joint3
      - rightarm_joint4
      - rightarm_joint5
      - rightarm_joint6
      - rightarm_joint7
    command_interfaces:
      - position
    state_interfaces:
      - position
      - velocity
```

6.4 修改dual_arm_description.ros2_control.xacro

```
<hardware>
    <!-- By default, set up controllers for simulation. This won't work on real
hardware -->
    <plugin>fake_components/GenericSystem</plugin>
</hardware>
```

6.5 新建`ompl_planning.yaml`文件

```
planner_configs:
  SBL:
    type: geometric::SBL
    range: 0.0  # Max motion added to tree. ==> maxDistance_ default: 0.0, if
0.0, set on setup()
  EST:
    type: geometric::EST
    range: 0.0  # Max motion added to tree. ==> maxDistance_ default: 0.0, if
0.0 setup()
```

```
    goal_bias: 0.05  # When close to goal select goal, with this probability.
default: 0.05
  LBKPIECE:
    type: geometric::LBKPIECE
    range: 0.0  # Max motion added to tree. ==> maxDistance_ default: 0.0, if
0.0, set on setup()
    border_fraction: 0.9  # Fraction of time focused on boarder default: 0.9
    min_valid_path_fraction: 0.5  # Accept partially valid moves above fraction.
default: 0.5
  BKPIECE:
    type: geometric::BKPIECE
    range: 0.0  # Max motion added to tree. ==> maxDistance_ default: 0.0, if
0.0, set on setup()
    border_fraction: 0.9  # Fraction of time focused on boarder default: 0.9
    failed_expansion_score_factor: 0.5  # When extending motion fails, scale
score by factor. default: 0.5
    min_valid_path_fraction: 0.5  # Accept partially valid moves above fraction.
default: 0.5
  KPIECE:
    type: geometric::KPIECE
    range: 0.0  # Max motion added to tree. ==> maxDistance_ default: 0.0, if
0.0, set on setup()
    goal_bias: 0.05  # When close to goal select goal, with this probability.
default: 0.05
    border_fraction: 0.9  # Fraction of time focused on boarder default: 0.9
(0.0,1.]
    failed_expansion_score_factor: 0.5  # When extending motion fails, scale
score by factor. default: 0.5
    min_valid_path_fraction: 0.5  # Accept partially valid moves above fraction.
default: 0.5
  RRT:
    type: geometric::RRT
    range: 0.0  # Max motion added to tree. ==> maxDistance_ default: 0.0, if
0.0, set on setup()
    goal_bias: 0.05  # When close to goal select goal, with this probability?
default: 0.05
  RRTConnect:
    type: geometric::RRTConnect
    range: 0.0  # Max motion added to tree. ==> maxDistance_ default: 0.0, if
0.0, set on setup()
  RRTstar:
    type: geometric::RRTstar
    range: 0.0  # Max motion added to tree. ==> maxDistance_ default: 0.0, if
0.0, set on setup()
    goal_bias: 0.05  # When close to goal select goal, with this probability?
default: 0.05
    delay_collision_checking: 1  # Stop collision checking as soon as C-free
parent found. default 1
  TRRT:
    type: geometric::TRRT
    range: 0.0  # Max motion added to tree. ==> maxDistance_ default: 0.0, if
0.0, set on setup()
    goal_bias: 0.05  # When close to goal select goal, with this probability?
default: 0.05
    max_states_failed: 10  # when to start increasing temp. default: 10
```

```
    temp_change_factor: 2.0  # how much to increase or decrease temp. default:
2.0
    min_temperature: 10e-10  # lower limit of temp change. default: 10e-10
    init_temperature: 10e-6  # initial temperature. default: 10e-6
    frountier_threshold: 0.0  # dist new state to nearest neighbor to disqualify
as frontier. default: 0.0 set in setup()
    frountierNodeRatio: 0.1  # 1/10, or 1 nonfrontier for every 10 frontier.
default: 0.1
    k_constant: 0.0  # value used to normalize expresssion. default: 0.0 set in
setup()
  PRM:
    type: geometric::PRM
    max_nearest_neighbors: 10  # use k nearest neighbors. default: 10
  PRMstar:
    type: geometric::PRMstar
  FMT:
    type: geometric::FMT
    num_samples: 1000  # number of states that the planner should sample.
default: 1000
    radius_multiplier: 1.1  # multiplier used for the nearest neighbors search
radius. default: 1.1
    nearest_k: 1  # use Knearest strategy. default: 1
    cache_cc: 1  # use collision checking cache. default: 1
    heuristics: 0  # activate cost to go heuristics. default: 0
    extended_fmt: 1  # activate the extended FMT*: adding new samples if planner
does not finish successfully. default: 1
  BFMT:
    type: geometric::BFMT
    num_samples: 1000  # number of states that the planner should sample.
default: 1000
    radius_multiplier: 1.0  # multiplier used for the nearest neighbors search
radius. default: 1.0
    nearest_k: 1  # use the Knearest strategy. default: 1
    balanced: 0  # exploration strategy: balanced true expands one tree every
iteration. False will select the tree with lowest maximum cost to go. default: 1
    optimality: 1  # termination strategy: optimality true finishes when the
best possible path is found. Otherwise, the algorithm will finish when the first
feasible path is found. default: 1
    heuristics: 1  # activates cost to go heuristics. default: 1
    cache_cc: 1  # use the collision checking cache. default: 1
    extended_fmt: 1  # Activates the extended FMT*: adding new samples if
planner does not finish successfully. default: 1
  PDST:
    type: geometric::PDST
  STRIDE:
    type: geometric::STRIDE
    range: 0.0  # Max motion added to tree. ==> maxDistance_ default: 0.0, if
0.0, set on setup()
    goal_bias: 0.05  # When close to goal select goal, with this probability.
default: 0.05
    use_projected_distance: 0  # whether nearest neighbors are computed based on
distances in a projection of the state rather distances in the state space
itself. default: 0
    degree: 16  # desired degree of a node in the Geometric Near-neightbor
Access Tree (GNAT). default: 16
```

```
    max_degree: 18  # max degree of a node in the GNAT. default: 12
    min_degree: 12  # min degree of a node in the GNAT. default: 12
    max_pts_per_leaf: 6  # max points per leaf in the GNAT. default: 6
    estimated_dimension: 0.0  # estimated dimension of the free space. default:
0.0
    min_valid_path_fraction: 0.2  # Accept partially valid moves above fraction.
default: 0.2
  BiTRRT:
    type: geometric::BiTRRT
    range: 0.0  # Max motion added to tree. ==> maxDistance_ default: 0.0, if
0.0, set on setup()
    temp_change_factor: 0.1  # how much to increase or decrease temp. default:
0.1
    init_temperature: 100  # initial temperature. default: 100
    frountier_threshold: 0.0  # dist new state to nearest neighbor to disqualify
as frontier. default: 0.0 set in setup()
    frountier_node_ratio: 0.1  # 1/10, or 1 nonfrontier for every 10 frontier.
default: 0.1
    cost_threshold: 1e300  # the cost threshold. Any motion cost that is not
better will not be expanded. default: inf
  LBTRRT:
    type: geometric::LBTRRT
    range: 0.0  # Max motion added to tree. ==> maxDistance_ default: 0.0, if
0.0, set on setup()
    goal_bias: 0.05  # When close to goal select goal, with this probability.
default: 0.05
    epsilon: 0.4  # optimality approximation factor. default: 0.4
  BiEST:
    type: geometric::BiEST
    range: 0.0  # Max motion added to tree. ==> maxDistance_ default: 0.0, if
0.0, set on setup()
  ProjEST:
    type: geometric::ProjEST
    range: 0.0  # Max motion added to tree. ==> maxDistance_ default: 0.0, if
0.0, set on setup()
    goal_bias: 0.05  # When close to goal select goal, with this probability.
default: 0.05
  LazyPRM:
    type: geometric::LazyPRM
    range: 0.0  # Max motion added to tree. ==> maxDistance_ default: 0.0, if
0.0, set on setup()
  LazyPRMstar:
    type: geometric::LazyPRMstar
  SPARS:
    type: geometric::SPARS
    stretch_factor: 3.0  # roadmap spanner stretch factor. multiplicative upper
bound on path quality. It does not make sense to make this parameter more than 3.
default: 3.0
    sparse_delta_fraction: 0.25  # delta fraction for connection distance. This
value represents the visibility range of sparse samples. default: 0.25
    dense_delta_fraction: 0.001  # delta fraction for interface detection.
default: 0.001
    max_failures: 1000  # maximum consecutive failure limit. default: 1000
  SPARStwo:
    type: geometric::SPARStwo
```

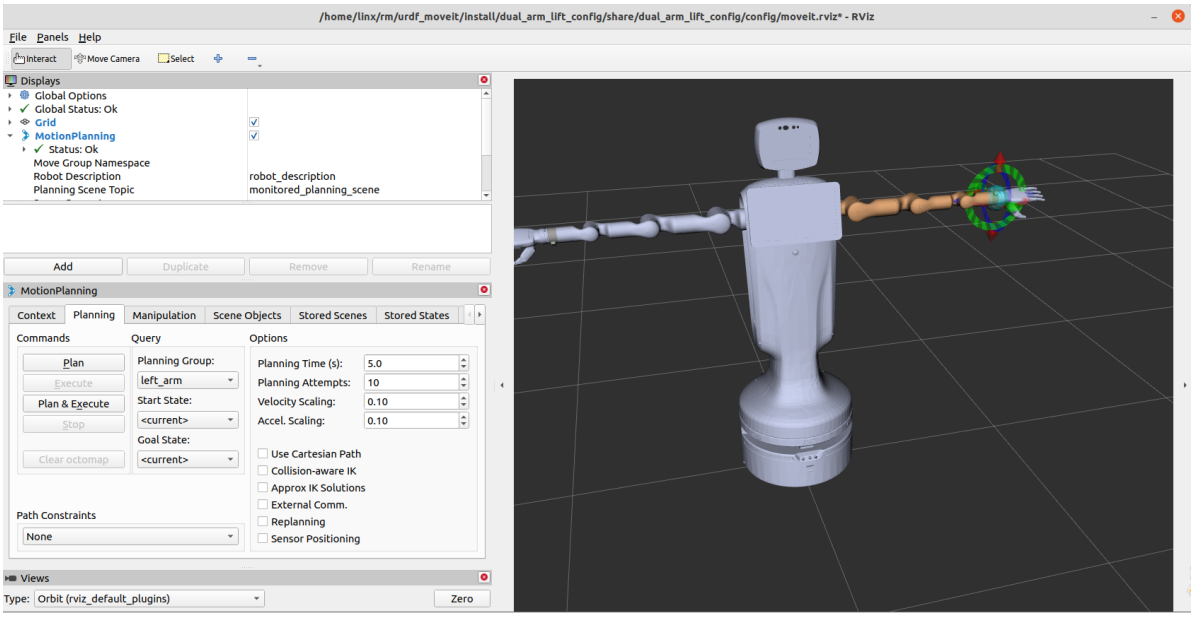
```
    stretch_factor: 3.0  # roadmap spanner stretch factor. multiplicative upper
bound on path quality. It does not make sense to make this parameter more than 3.
default: 3.0
    sparse_delta_fraction: 0.25  # delta fraction for connection distance. This
value represents the visibility range of sparse samples. default: 0.25
    dense_delta_fraction: 0.001  # delta fraction for interface detection.
default: 0.001
    max_failures: 5000  # maximum consecutive failure limit. default: 5000
left_arm:
  default_planner_config: RRTConnect
  planner_configs:
    - SBL
    - EST
    - LBKPIECE
    - BKPIECE
    - KPIECE
    - RRT
    - RRTConnect
    - RRTstar
    - TRRT
    - PRM
    - PRMstar
    - FMT
    - BFMT
    - PDST
    - STRIDE
    - BiTRRT
    - LBTRRT
    - BiEST
    - ProjEST
    - LazyPRM
    - LazyPRMstar
    - SPARS
    - SPARStwo
  projection_evaluator: joints(l_joint1,l_joint2)
  longest_valid_segment_fraction: 0.05
right_arm:
  default_planner_config: RRTConnect
  planner_configs:
    - SBL
    - EST
    - LBKPIECE
    - BKPIECE
    - KPIECE
    - RRT
    - RRTConnect
    - RRTstar
    - TRRT
    - PRM
    - PRMstar
    - FMT
    - BFMT
    - PDST
    - STRIDE
    - BiTRRT
```

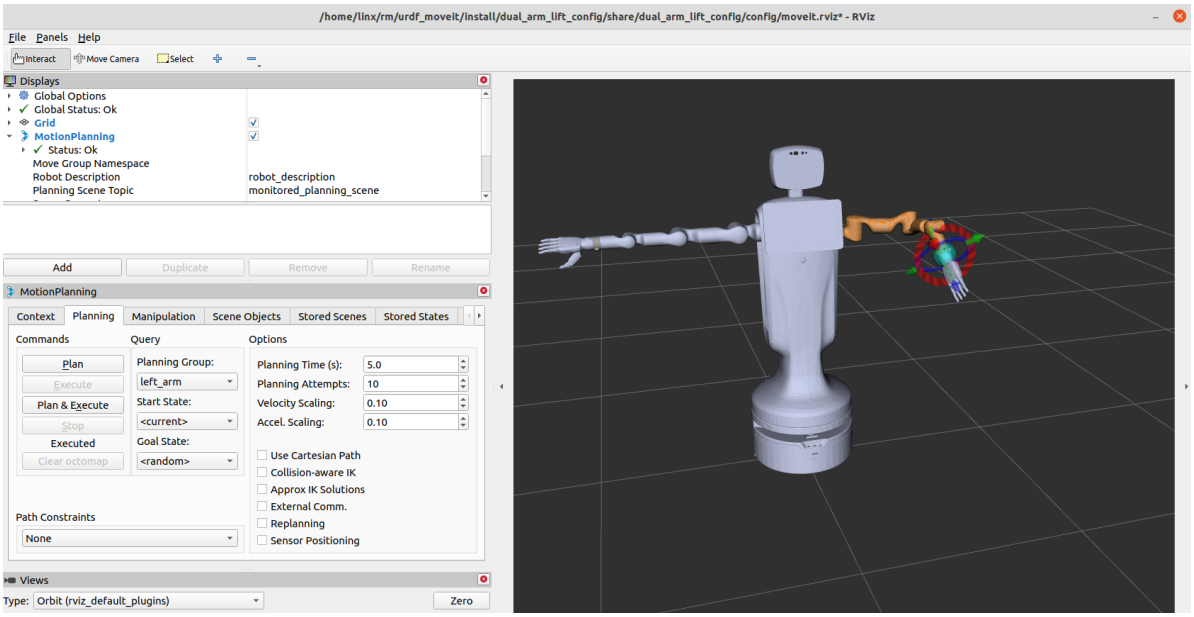
```
    - LBTRRT
    - BiEST
    - ProjEST
    - LazyPRM
    - LazyPRMstar
    - SPARS
    - SPARStwo
  projection_evaluator: joints(r_joint1,r_joint2)
  longest_valid_segment_fraction: 0.05
```

这里需要注意的是`moveit_controllers.yaml`、`ros2_controllers.yaml`中的规划器名和所包含关节名必须与模型中的关节名保持一致。其余如关节角度限制、初始角度设置等配置文件也须逐一检查是否匹配。

6.6 编译测试：

```
# 编译
colcon build
# 测试
source install/setup.bash
ros2 launch dual_arm_lift_config demo.launch.py
```





如果觉得规划或者执行的速度慢，可以修改`joint_limits.yaml`里面的参数，或者直接修改`rviz2`中的`Velocity Scaling`和`Accel Scaling`两个参数

7. 控制真实的机械臂

我们的需求是需要使用Moveit2控制真实的机械臂，双规划组与实际机械臂进行通讯控制时，应当同时运行两套driver和control，以不同命名空间区分。二者的driver连接参数的连接ip和udp回传ip必须错开，否则会出现一条臂不断闪现另一条臂状态的情况。

在功能包`dual_rm_control`为实现Moveit2控制真实机械臂时所必须的一个功能包，该功能包的主要作用机器人控制器，将 Moveit2 规划的机械臂轨迹，通过三次样条插值细分，按照`20ms`的控制周期发给`dual_rm_driver`节点

`dual_rm_control`这个的功能包`launch`目录中新建文件`dual_arm_control.launch.py`启动两套`control`，内容如下：

```
from launch import LaunchDescription
from launch_ros.actions import Node
def generate_launch_description():
    ld = LaunchDescription()
    left_control_node = Node(
    package='dual_rm_control', #节点所在的功能包
    executable='dual_rm_control', #表示要运行的可执行文件名或脚本名字.py
    parameters= [
                    {'follow': True},
                    {'arm_type': 75}
                ],              #接入参数文件
    output='screen', #用于将话题信息打印到屏幕
    namespace='left_arm_controller'
    )

    right_control_node = Node(
    package='dual_rm_control', #节点所在的功能包
    executable='dual_rm_control', #表示要运行的可执行文件名或脚本名字.py
    parameters= [
                    {'follow': True},
                    {'arm_type': 75}
                ],              #接入参数文件
```

```
    output='screen', #用于将话题信息打印到屏幕
    namespace='right_arm_controller'
    )

    ld.add_action(left_control_node)
    ld.add_action(right_control_node)
    return ld
```

`dual_rm_driver`功能包是机械臂的底层驱动程序，订阅和发布各`topic`数据，更新`RVIZ`内机械臂各关节角度

同样`dual_rm_driver`这个的功能包`launch`目录中新建文件`dual_arm_driver.launch.py`启动两套`driver`，从而控制双臂，内容如下。

```
import launch
import os
import yaml
import launch_ros
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.substitutions import Command, LaunchConfiguration
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    ld = LaunchDescription()

    left_arm_config =
os.path.join(get_package_share_directory('dual_rm_driver'), 'config',
'dual_left_config.yaml')
    right_arm_config =
os.path.join(get_package_share_directory('dual_rm_driver'), 'config',
'dual_right_config.yaml')

    with open(left_arm_config, 'r') as f:
        left_arm_params = yaml.safe_load(f)["left_arm_controller/rm_driver"]
["ros__parameters"]

    with open(right_arm_config, 'r') as f:
        right_arm_params = yaml.safe_load(f)["right_arm_controller/rm_driver"]
["ros__parameters"]

    left_arm_driver = Node(
        package="dual_rm_driver",
        executable="dual_rm_driver",
        parameters=[left_arm_config],
        output="screen",
        namespace='left_arm_controller'
    )

    right_arm_driver = Node(
        package="dual_rm_driver",
        executable="dual_rm_driver",
        parameters=[right_arm_config],
        output="screen",
        namespace='right_arm_controller'
```

```
    )

    ld.add_action(left_arm_driver)
    ld.add_action(right_arm_driver)

    return ld
```

在`dual_arm_lift_config`功能包的`launch`目录下新建控制真实机械臂的启动文件`real_moveit_demo.launch.py`内容如下。

```
import os
import yaml
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, TimerAction,
IncludeLaunchDescription
from launch.substitutions import LaunchConfiguration
from launch.conditions import IfCondition, UnlessCondition
from launch_ros.actions import Node
from launch.actions import ExecuteProcess
from ament_index_python.packages import get_package_share_directory
from launch.launch_description_sources import PythonLaunchDescriptionSource
import xacro

def load_file(package_name, file_path):
    package_path = get_package_share_directory(package_name)
    absolute_file_path = os.path.join(package_path, file_path)

    try:
        with open(absolute_file_path, "r") as file:
            return file.read()
    except EnvironmentError:  # parent of IOError, OSError *and* WindowsError
where available
        return None

def load_yaml(package_name, file_path):
    package_path = get_package_share_directory(package_name)
    absolute_file_path = os.path.join(package_path, file_path)

    try:
        with open(absolute_file_path, "r") as file:
            return yaml.safe_load(file)
    except EnvironmentError:  # parent of IOError, OSError *and* WindowsError
where available
        return None

def generate_launch_description():

    # planning_context
    robot_description_config = xacro.process_file(
        os.path.join(
            get_package_share_directory("dual_arm_lift_config"),
            "config",
            "dual_arm_description.urdf.xacro",
        )
    )
```

```
    robot_description = {"robot_description": robot_description_config.toxml()}

    robot_description_semantic_config = load_file(
        "dual_arm_lift_config", "config/dual_arm_description.srdf"
    )

    robot_description_semantic = {
        "robot_description_semantic": robot_description_semantic_config
    }

    kinematics_yaml = load_yaml(
        "dual_arm_lift_config", "config/kinematics.yaml"
    )
    robot_description_kinematics = {"robot_description_kinematics":
kinematics_yaml}

    joint_limits_yaml = load_yaml("dual_arm_lift_config",
"config/joint_limits.yaml")
    joint_limits = {'robot_description_planning': joint_limits_yaml}

    # Planning Functionality
    ompl_planning_pipeline_config = {
        "move_group": {
            "planning_plugin": "ompl_interface/OMPLPlanner",
            "request_adapters":
"""default_planner_request_adapters/AddTimeOptimalParameterization
default_planner_request_adapters/FixWorkspaceBounds
default_planner_request_adapters/FixStartStateBounds
default_planner_request_adapters/FixStartStateCollision
default_planner_request_adapters/FixStartStatePathConstraints""",
            "start_state_max_bounds_error": 0.1,
        }
    }
    ompl_planning_yaml = load_yaml(
        "dual_arm_lift_config", "config/ompl_planning.yaml"
    )
    ompl_planning_pipeline_config["move_group"].update(ompl_planning_yaml)

    # Trajectory Execution Functionality
    moveit_simple_controllers_yaml = load_yaml(
        "dual_arm_lift_config", "config/moveit_controllers.yaml"
    )

    moveit_controllers = {
        "moveit_simple_controller_manager": moveit_simple_controllers_yaml,
        "moveit_controller_manager":
"moveit_simple_controller_manager/MoveItSimpleControllerManager",
    }

    trajectory_execution = {
        "moveit_manage_controllers": True,
        "trajectory_execution.allowed_execution_duration_scaling": 1.2,
        "trajectory_execution.allowed_goal_duration_margin": 0.5,
        "trajectory_execution.allowed_start_tolerance": 0.1,
```

```
    }

    planning_scene_monitor_parameters = {
        "publish_planning_scene": True,
        "publish_geometry_updates": True,
        "publish_state_updates": True,
        "publish_transforms_updates": True,
        "use_fake_hardware": False,
        "publish_robot_description": True,
        "publish_robot_description_semantic": True,
    }

    # Start the actual move_group node/action server
    run_move_group_node = Node(
        package="moveit_ros_move_group",
        executable="move_group",
        output="screen",
        parameters=[
            robot_description,
            robot_description_semantic,
            kinematics_yaml,
            joint_limits,
            ompl_planning_pipeline_config,
            trajectory_execution,
            moveit_controllers,
            planning_scene_monitor_parameters,
        ],
    )

    # RViz
    rviz_base =
os.path.join(get_package_share_directory("dual_arm_lift_config"), "config")
    rviz_full_config = os.path.join(rviz_base, "moveit.rviz")

    rviz_node = Node(
        package="rviz2",
        executable="rviz2",
        name="rviz2",
        output="log",
        arguments=["-d", rviz_full_config],
        parameters=[
            robot_description,
            robot_description_semantic,
            ompl_planning_pipeline_config,
            kinematics_yaml,
        ],
    )

    delayed_rviz_node = TimerAction(
        period=40.0,
        actions=[rviz_node]
    )

    # Static TF
    static_tf = Node(
```

```
        package="tf2_ros",
        executable="static_transform_publisher",
        name="static_transform_publisher",
        output="log",
        arguments=["0.0", "0.0", "0.0", "0.0", "0.0", "0.0", "world",
"base_link"],
    )

    # Publish TF
    robot_state_publisher = Node(
        package="robot_state_publisher",
        executable="robot_state_publisher",
        name="robot_state_publisher",
        output="both",
        parameters=[robot_description],
    )

    # ros2_control using FakeSystem as hardware
    ros2_controllers_path = os.path.join(
        get_package_share_directory("dual_arm_lift_config"),
        "config",
        "ros2_controllers.yaml",
    )

    ros2_control_node = Node(
        package="controller_manager",
        executable="ros2_control_node",
        parameters=[robot_description, ros2_controllers_path],
        output={
            "stdout": "screen",
            "stderr": "screen",
        },
    )

    # Load controllers
    load_controllers = []
    for controller in ["left_arm_controller"]:
        load_controllers += [
            ExecuteProcess(
                cmd=["ros2 run controller_manager spawner.py
{}".format(controller)],
                shell=True,
                output="screen",
            )
        ]
    load_controllers2 = []
    for controller in ["right_arm_controller"]:
        load_controllers += [
            ExecuteProcess(
                cmd=["ros2 run controller_manager spawner.py
{}".format(controller)],
                shell=True,
                output="screen",
            )
        ]
```

```
    # Warehouse mongodb server
    mongodb_server_node = Node(
       package="warehouse_ros_mongo",
       executable="mongo_wrapper_ros.py",
       parameters=[
           {"warehouse_port": 33829},
           {"warehouse_host": "localhost"},
           {"warehouse_plugin": "warehouse_ros_mongo::MongoDatabaseConnection"},
       ],
       output="screen",
    )

    # Include dual_arm_driver.launch.py
    dual_arm_driver_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(
                get_package_share_directory('dual_rm_driver'),
                'launch',
                'dual_arm_driver.launch.py'
            )
        )
    )

    # Include dual_arm_control.launch.py
    dual_arm_control_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(
                get_package_share_directory('dual_rm_control'),
                'launch',
                'dual_arm_control.launch.py'
            )
        )
    )

    return LaunchDescription(
        [
            delayed_rviz_node,
            static_tf,
            robot_state_publisher,
            run_move_group_node,
            ros2_control_node,
            dual_arm_driver_launch,
            dual_arm_control_launch,
        ] + load_controllers + load_controllers2
    )
```

这里需要注意的是`dual_rm_driver`功能包目录下的`config`里面左臂和右臂的配置文件的关节必须跟`URDF`保持一致，否则`UDP`上报的时候找不到相应的关节，会导致`Rviz`中的机器人跟实际的不符。

```
# 左臂
arm_joints: [leftarm_joint1, leftarm_joint2, leftarm_joint3, leftarm_joint4,
leftarm_joint5, leftarm_joint6, leftarm_joint7]
# 右臂
arm_joints: [rightarm_joint1, rightarm_joint2, rightarm_joint3, rightarm_joint4,
rightarm_joint5, rightarm_joint6, rightarm_joint7]
```

编译测试

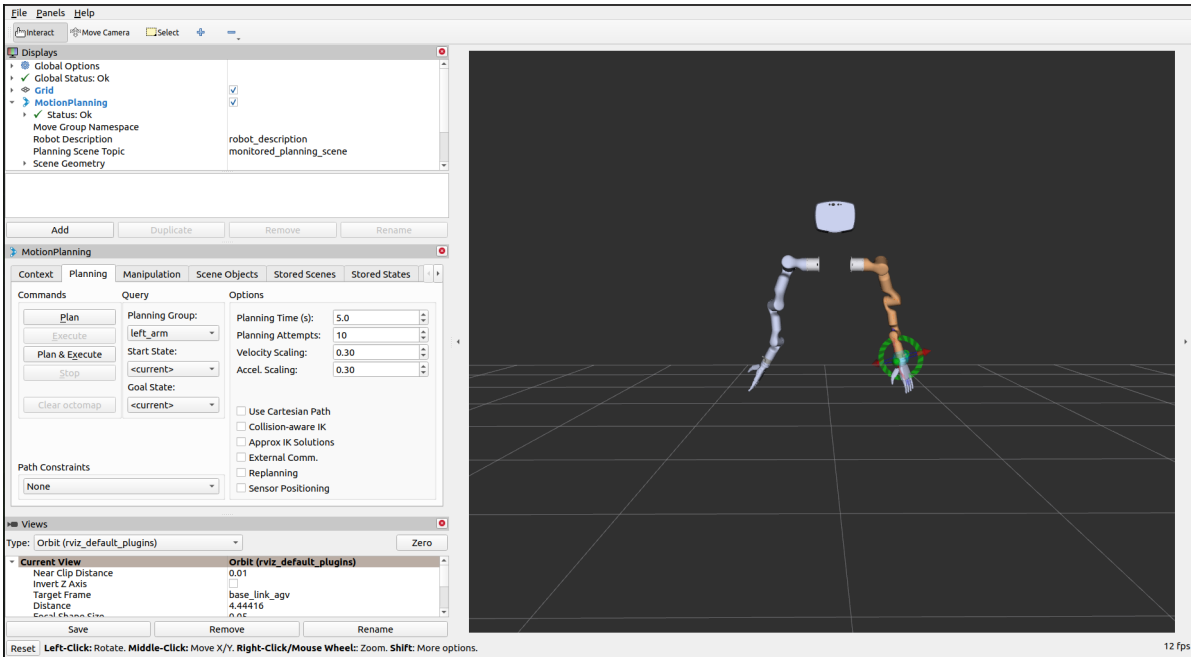
```
# 编译
colcon build

source install/setup.bash
ros2 launch dual_arm_lift_config real_moveit_demo.launch.py
```

这里测试的时候由于加载的STL模型过大，导致加载很慢，为了方便测试，可以注释`URDF`中暂时用不到的`STL`文件，然后你就会发现启动极其丝滑。

```
140     delayed_rviz_node = TimerAction(
141         period=4.0,
142         actions=[rviz_node]
143     )
```

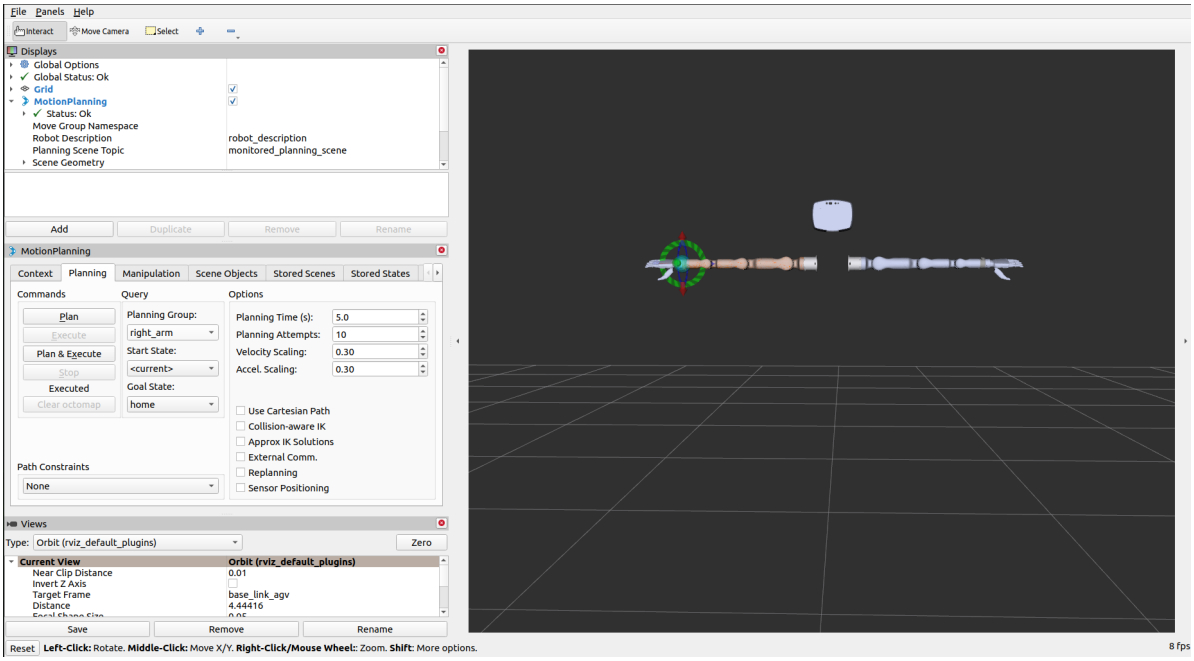
以上内容是修改启动的加载时间。

启动Moveit2!

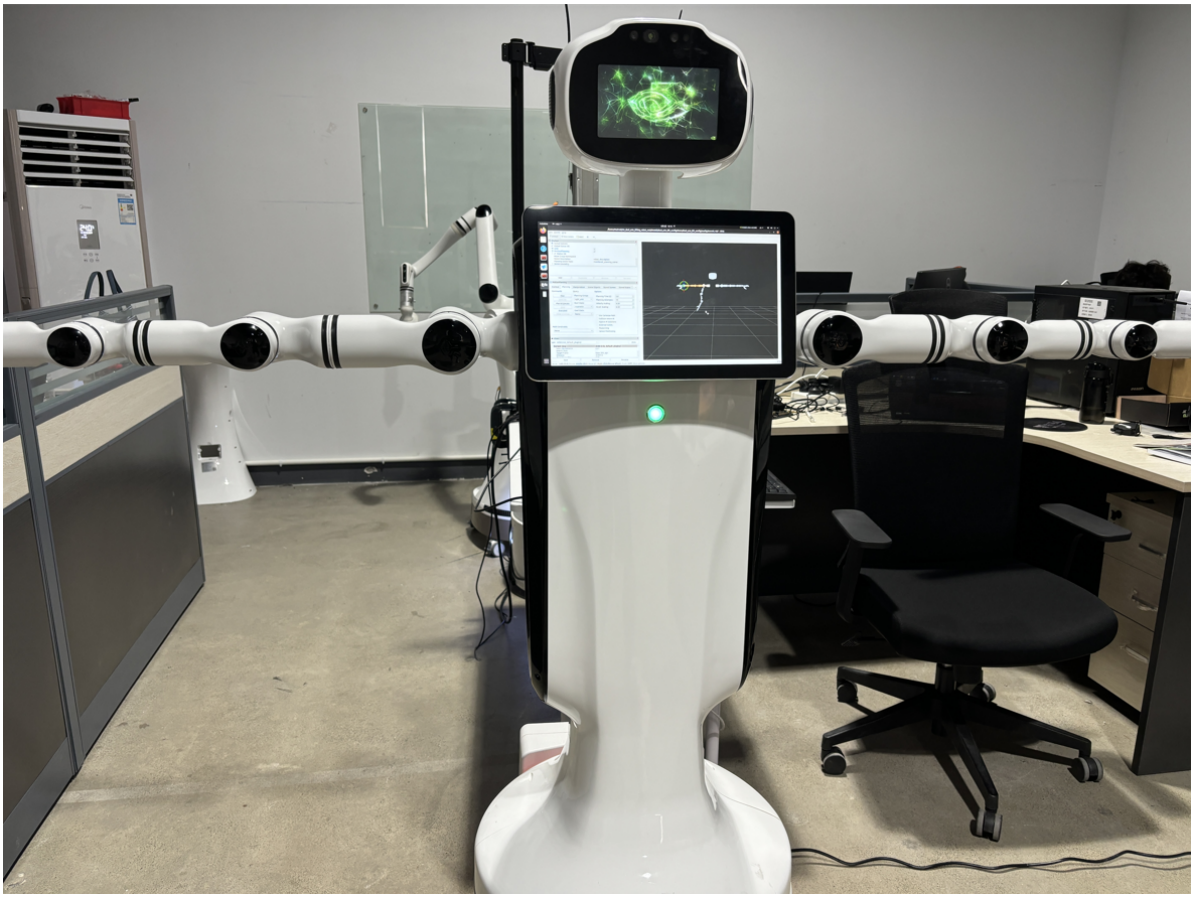


真实机械臂状态：

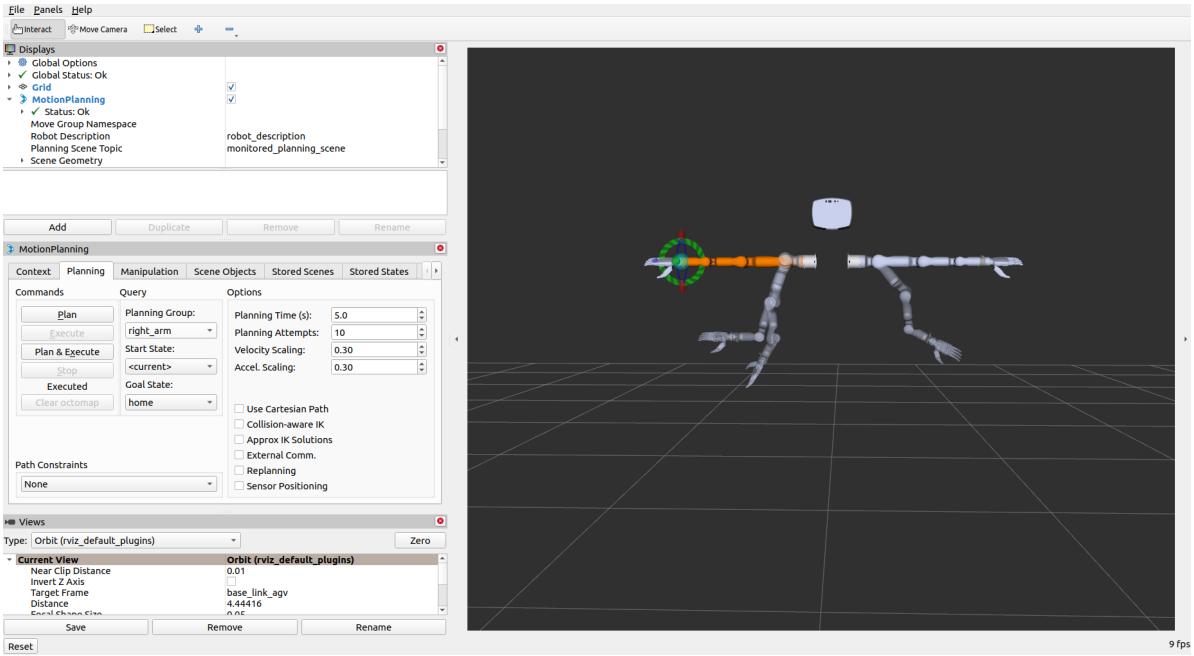
回到home姿态



真实机械臂状态：



按住机械臂示教按钮，查看机械臂的状态是否反馈到`ROS2`中，`Rviz2`中可查看状态。



左右臂的虚影就是机械臂的目标状态

真实机械状态

OK，到此所有功能一些正常。